AI-POWERED FILE ORGANIZER APPLICATION

MR. SARUN KHUMTHAI
MR. SUPAKORN TUNGPATOMPRAMOTE
MR. PUWADECH INTONG

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR
THE DEGREE OF BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)
FACULTY OF ENGINEERING
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2025

AI-powered file organizer application

Mr. Sarun Khumthai
Mr. Supakorn Tungpatompramote
Mr. Puwadech Intong

A Project Submitted in Partial Fulfillment
of the Requirements for
the Degree of Bachelor of Engineering (Computer Engineering)
Faculty of Engineering
King Mongkut's University of Technology Thonburi
2025

Project Committee

. . . . . Project Advisor
(Suthathip Maneewongvatana, Ph.D.)

. . . . . Committee Member
(Jaturon Harnsomburana, Ph.D.)

. . . . . Committee Member
(Naruemon Wattanapongsakorn, Ph.D.)

Project Title	AI-powered file organizer application
Credits	3
Member(s)	Mr. Sarun Khumthai
	Mr. Supakorn Tungpatompramote
	Mr. Puwadech Intong
Project Advisor	Suthathip Maneewongvatana, Ph.D.
Program	Bachelor of Engineering
Field of Study	Computer Engineering
Department	Computer Engineering
Faculty	Engineering
Academic Year	2025

Abstract

The project, “AI-powered file organizer application”, is motivated by the fact that nowadays almost everyone owns a computer and inevitable to create or handle “files.” The number of them is enormous, managing them can become troublesome. This project is going to address this common problem by developing a desktop application that can automatically read and understand file’s content to organize files with more precision. Our objective then is for this solution to assist with and reduce the burden of manual file management. Furthermore, aside from organization, it may also integrate AI to perform more advanced actions which aims to establish a foundation for agentic systems related to file management such as summarization, note taking, searching or calendar taking. Everything must be done locally, bringing greater convenience and ease to users. The performance evaluation of the “AI-powered file organizer application” showed that the overall performance was at the level of effectively solving the problem with minor improvements possible (mean score of 4.31) and standard deviation was 0.19, indicating that the experts’ opinions regarding the application’s performance were similar and consistent. The experts also provided overall comments on the “AI-powered file organizer application”, stating that the application has academic value and can be applied in real-world use. It can be further developed as an Open-Source Project. The application has a very strong foundation and is ready for personal use or small teams.

Keywords: AI-powered file organizer application / File organization / Classification / LLM / RAG

หัวข้อปริญญานิพนธ์	แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI
หน่วยกิต	3
ผู้เขียน	นายศรัณย์ คำไทย
	นายศุภกร ตั้งปฐมปราโมทย์
	นายภูวเดช อินทอง
อาจารย์ที่ปรึกษา	ดร. สุธาทิพย์ มณีวงศ์วัฒนา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมคอมพิวเตอร์
ภาควิชา	วิศวกรรมคอมพิวเตอร์
คณะ	วิศวกรรมศาสตร์
ปีการศึกษา	2568

บทคัดย่อ

โครงงาน "แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI" มีแรงจูงใจมาจากข้อเท็จจริงที่ว่า ในปัจจุบันแทบทุกคนมีคอมพิวเตอร์และหลีกเลี่ยงไม่ได้ที่จะต้องสร้างหรือจัดการ "ไฟล์" ต่าง ๆ ซึ่งจำนวนไฟล์เหล่านี้มีปริมาณมาก ทำให้การจัดการไฟล์กลายเป็นเรื่องยุ่งยาก ตัวโครงงานจึงมุ่งแก้ไขปัญหาที่พบได้บ่อยนี้ โดยการพัฒนาแอปพลิเคชันเดสก์ท็อปที่สามารถอ่านและเข้าใจเนื้อหาของไฟล์ได้โดยอัตโนมัติ เพื่อที่จัดระเบียบไฟล์ที่ได้รับเข้ามาใหม่อย่างมีความแม่นยำมากขึ้น วัตถุประสงค์ของโครงงานคือการช่วยเหลือผู้ใช้ในการลดภาระจากการจัดการไฟล์ด้วยตนเอง โดยมีการผสานความสามารถของ AI เพื่อดำเนินงานขั้นสูงซึ่งช่วยวางรากฐานสำหรับระบบเชิง Agentic ที่เกี่ยวข้องกับการจัดการไฟล์ เช่น การสรุปเนื้อหา การจดบันทึก การค้นหา หรือการบันทึกลงปฏิทิน ทั้งหมดนี้จะทำงานภายในเครื่องแบบ Local เพื่อความสะดวกและความง่ายในการใช้งานของผู้ใช้ หลังจากได้พัฒนาตัวโปรแกรมเสร็จสิ้นแล้ว ผู้จัดทำได้ทำการประเมินประสิทธิภาพของ "แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI" ด้วยแบบประเมินประสิทธิภาพโดยผู้เชี่ยวชาญ ซึ่งผลลัพธ์คือ ภาพรวมตัวแอปพลิเคชันอยู่ในระดับที่แก้ไขปัญหาได้อย่างมีประสิทธิภาพ อาจปรับปรุงเล็กน้อยได้ (ค่าเฉลี่ยเท่ากับ 4.31) ค่าเบี่ยงเบนมาตรฐานมีค่าเท่ากับ 0.19 หมายความว่า มีการกระจ่ายตัวจากค่าเฉลี่ยต่ำ หรือก็คือ ผู้เชี่ยวชาญมีความคิดเห็นเกี่ยวกับประสิทธิภาพของแอปพลิเคชันใกล้เคียงและสอดคล้องกัน ทั้งนี้ยังให้ความคิดเห็นโดยรวมเกี่ยวกับ "แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI" ว่า แอปพลิเคชันนี้มีคุณค่าทางวิชาการและสามารถนำไปใช้งานจริงได้ สามารถพัฒนาต่อยอดเป็น Open Source Project ตัวแอปพลิเคชันมีพื้นฐานที่ดีมากและพร้อมใช้งานสำหรับ personal use หรือ small team

คำสำคัญ: แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI / การจัดการไฟล์ / การจำแนกเอกสาร / LLM / RAG

ACKNOWLEDGMENTS

This project would not have been possible with the support of many people. Firstly, we would like to express our sincere gratitude to our advisor, Suthathip Maneewongvatana, Ph.D., for her guidance, time, and support throughout this project. We would also like to thank every committee member who gives us valuable insights and feedback alongside our advisor throughout the entire semester, which is crucial to the development and success of this project.

A special thanks goes to three experts, Saenguthai Mortho, Asst. Ph.D., Mr. Sonthaya Kueakhiri and Mr. Theerawat NaNakhon, who give their time and expertise to us for the Expert Evaluation form and its feedback.

Lastly, we would like to thank everyone involved in the making of this project including those who cheer us on for their constant support and encouragement. Thank you from the bottom of our heart.

CONTENTS

PAGE

LIST OF TABLES

TABLE **PAGE**

LIST OF FIGURES

FIGURE **PAGE**

CHAPTER 1 INTRODUCTION

1.1 Motivation

At present, as the world enters the information age, technological knowledge has become widespread to all. Almost everyone owns at least one computing device. As a result, the creation and storage of digital data have increased continuously and on a massive scale. Users may have to deal with various types of files, such as documents (pdf, .docx, .txt), images (.png, .jpg), presentations (.pptx), and spreadsheets (.xlsx), which are ones that are widely used and exchanged in daily life. Each user has a different way of organizing files, but a common problem is that they have to move files manually after downloading them. If there are only a few files, this may not be a major issue. However, when the number of files increases, manual file management becomes a repetitive and time-consuming process prone to errors. Leaving files accumulated in the Downloads folder is not an appropriate method. Therefore, we aim to develop an automated file management system that can read and understand file contents, analyze their meaning, and accurately categorize files. The system will use AI capabilities, particularly LLM, to move files to the appropriate destination folders.

1.2 Objective

- To develop a desktop application that helps users manage files efficiently with AI.
- To reduce repetitive tasks and errors caused by manual file management.
- To improve the convenience of searching and organizing files through customizable file management templates.
- To establish a foundation for future expansion toward Agentic AI features, such as file content summarization, note generation, and calendar integration.

1.3 Project Scope

The project is a desktop application that automatically manages files by using an LLM to analyze contents and move files to appropriate folders. The system must be able to:

- Support reading and analyzing PDF, Word, Excel, PowerPoint, TXT, and image files.
- Automatically categorize files and move them to designated folders based on templates.
- Provide a Preview system that allows users to review the actions before approval.
- Record operation history and support Undo/Redo for every change.
- Provide a user-friendly and secure UI, running on Windows.

CHAPTER 2 BACKGROUND, THEORIES, AND RELATED RESEARCH

2.1 Related Theories

2.1.1 Natural Language Processing (NLP)

Natural Language Processing, or NLP, is a field of artificial intelligence concerned with enabling computers to understand, analyze, and generate human-like language. NLP has various applications, such as text classification, information extraction, and summarization. In this project, NLP is applied to analyze and classify document files based on the content contained within each file.

2.1.2 Large Language Models (LLM)

Large Language Models, or LLMs, are large-scale language models trained on massive datasets to process and generate text in a way that is similar to human language. Examples of LLMs include GPT, Llama, Gemma, and others. The main strength of LLMs is their ability to understand textual context and generate complex outputs, such as document classification, text summarization, or generating new file names based on specified conditions. In this project, the LLM is used and controlled through prompt-based instructions.

2.1.3 Vision Language Models (VLM)

Vision Language Models, or VLMs, are AI systems created by combining Large Language Models with vision encoders, allowing LLMs to “see” and understand visual information. With this capability, VLMs can process and provide advanced understanding of videos, images, and text inputs provided in prompts in order to generate text-based responses.

Unlike traditional Computer Vision models, VLMs are not limited to fixed classes or specific tasks such as classification or object detection. VLMs trained on large-scale text corpora and image/video-caption pairs can be instructed using natural language and applied to many classical vision tasks, as well as new generative AI tasks such as visual summarization and answering visual question.

2.1.4 Embedding Models

Embedding Models are AI models used to convert unstructured data, such as text, images, or documents, into numerical vector representations. These vectors capture the semantic meaning of the original data, allowing computers to compare, search, and analyze information based on meaning rather than exact keyword matching. For example, two documents that use different words but discuss similar topics may have similar vector representations. This makes embedding models useful for tasks such as semantic search, document similarity comparison, file recommendation, duplicate detection, and automatic file categorization.

2.1.5 GGUF model

.gguf AI model is a compressed model file format commonly used to run LLMs locally with tools such as llama.cpp. It stores model weights, tokenizer data, and metadata in one file, making it easy to load and use offline. GGUF also supports quantization, which reduces model size and memory usage, allowing AI models to run on normal computers with limited hardware. In this case, GGUF models are useful because they allow the system to analyze, classify, summarize, and rename files locally without sending user data to cloud services. This improves privacy and supports offline AI processing.

2.1.6 Vector Database

A Vector Database is a specialized database designed to store, manage, and search data in the form of high-dimensional vectors. These vectors can capture the conceptual meaning of unstructured data, such as text, images, or audio. Instead of relying only on exact keyword matching like traditional databases, a vector database focuses on semantic similarity search. This allows the system to efficiently retrieve information that is meaningfully related to a query.

2.1.7 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, is a technique or process used to improve the accuracy and reliability of LLMs by retrieving additional information from trusted external sources beyond the model's training data before generating a final response. RAG is commonly used together with vector databases, which store information in numerical vector forms that AI systems can understand and retrieve efficiently.

2.1.8 Prompt Engineering

Prompt Engineering is the process of designing instruction text that is sent to an LLM in order to control the output and make it follow the desired format. For example, prompts can instruct an LLM to respond in JSON format or use few-shot examples so that the model can learn the expected pattern from sample inputs and outputs. This technique allows existing models to be used without additional fine-tuning. It is suitable for this project because the system requires accurate and clearly structured outputs, also known as structured output.

2.1.9 Hybrid Search

Hybrid Search is a search approach that combines two or more search techniques to produce more accurate and relevant results. In general, it combines keyword search, which focuses on exact word matching, with semantic search, which understands the context and meaning of text. This approach allows the system to search based on both exact words and related concepts. It can be used to help users find files more effectively by supporting various types of search queries according to user needs.

2.1.10 AI Agents

An AI Agent is a system that uses an LLM to control tasks on behalf of users. It can make decisions and perform actions automatically under defined conditions. The main strength of an AI Agent is its ability to select appropriate tools, such as calling APIs or accessing databases, and perform repetitive tasks efficiently. The main components of an AI Agent include Model, Tools, and Instructions. The Model is responsible for reasoning and decision-making, Tools are used to access information or perform actions, and Instructions define the behavior of the Agent to ensure that it operates according to the intended requirements.

2.2 Related Computer Technologies

2.2.1 Desktop applications

A desktop application is a software program installed and run directly on a personal computer or laptop, such as on Windows, macOS, or Linux, to perform specific tasks without always requiring a web browser.

2.2.2 Tauri

Tauri is a framework for developing lightweight desktop applications that can run on both Windows and macOS. It uses web technologies such as HTML, CSS, and JavaScript/TypeScript to build the user interface,

while Rust is used as the backend to connect with the file system and native APIs of the operating system. Tauri is suitable for applications that require speed, security, and a small application size.

2.2.3 React + TypeScript

React and TypeScript are used to develop the user interface of the application. React provides efficient state management. TypeScript improves code reliability by adding static type checking, which helps reduce development errors and makes the application easier to maintain.

2.2.4 Rust / Python

Rust is used to interact with the file system, manage batch processing, and handle tasks that require high performance and memory safety.
Python is used for AI-related processing, including running AI models and LLMs tasks. It is suitable for text processing, document analysis, file content extraction, and integration with machine learning libraries.

2.2.5 llama.ccp

llama.cpp is an open-source C/C++ project used to run Large Language Models locally on a computer. Its main goal is to enable LLM inference with minimal setup and good performance across different hardware. It is commonly used with GGUF models, which are model files optimized for local inference. With llama.cpp, users can run AI models without relying on cloud services, making them suitable for offline and privacy-focused applications. Hugging Face also documents GGUF usage with llama.cpp for downloading and running compatible models.

2.3 Related Research

2.3.1 Auto Organizer: A Machine Learning-Based Tool for Automatic Organization of Files

Sengar, Fatima, and Yadav proposed Auto Organizer, a machine learning-based tool for automatically organizing files on a digital device. The system was designed to solve the common problem of users having large numbers of downloaded or created files, which makes it difficult to locate specific files later. The proposed tool supports multiple file types, including PDF files, documents, images, and audio files. It applies different deep learning algorithms to classify files into predefined categories, with a total of twenty classification categories included in the system. In addition to file classification, the tool provides practical features such as directory sorting, file name recommendation, and file save path recommendation through a user-friendly graphical interface.
Relevance to the project : The basic of paper is very similar to the project “AI-powered file organizer application”, it may have some differences, but it is solid evidence that the solution to use AI models as a classifier to file organization problem is appropriate. In another sense, project “AI-powered file organizer application” is an extension and an upgrade to this paper with use of more advanced LLM models instead of simple fine-tuned machine learning.

2.3.2 Semantic File Systems

Gifford et al. introduced the Semantic File System, a file management approach that provides associative access to files by automatically extracting attributes from file contents using file-type-specific transducers. Unlike traditional hierarchical file systems, which require users to remember exact folder locations, semantic file systems allow users to locate files based on meaningful attributes such as author, title, file type, text content,

or program functions. The system also introduces virtual directories, where directory names are interpreted as queries and the results are displayed like normal folders. This design allows semantic access to remain compatible with existing file system tools and protocols. The study shows that semantic file systems can improve information retrieval and provide a more flexible storage abstraction than conventional tree-structured file systems. This research is relevant to the proposed automatic file organizer because it demonstrates the importance of content-based analysis, automatic indexing, and semantic metadata extraction for improving file discovery and organization.

Relevance to the project : To project “AI-powered file organizer application”, this paper can support the idea that file organization should not depend only on folder hierarchy or file extension. The paper provides an early theoretical and practical foundation for automatically extracting file meaning and using that information for retrieval. The project can be positioned as a modern extension of this idea by using machine learning, OCR, embeddings, or LLM-based summarization to classify and organize files automatically.

CHAPTER 3 METHODOLOGY

3.1 Project Details

This project aims to develop a desktop application that uses an LLM to automatically read, analyze, and categorize document files such as .pdf, .docx, .pptx, .xlsx and image files. The system needs to help reduce the burden of manual file management, such as moving files, renaming files, and creating new folders. Users can define their own file organization templates and review the results through a Preview screen before confirming the actual operation. The application is also designed with a History/Undo system to ensure that any changes made by the system can be reversed when needed. In addition, the application includes secondary features that apply Agentic AI concepts, such as file summarization, schedule generation, and a note-taking workspace for folders.

System Requirements

- The system must be able to read and support document files, including .pdf, .docx, .pptx, .xlsx and image files.
- The system must be able to use AI to analyze and classify files based on the content.
- The system must be able to create destination folder paths.
- The system must provide a Preview function that allows users to review the results before confirming the actual operation.
- The system must record all operation history and support Undo/Redo.
- The system must provide a user-friendly and easy-to-understand UI to improve user convenience.

3.2 System Overview

3.2.1 Dataflow Diagram (level 0)

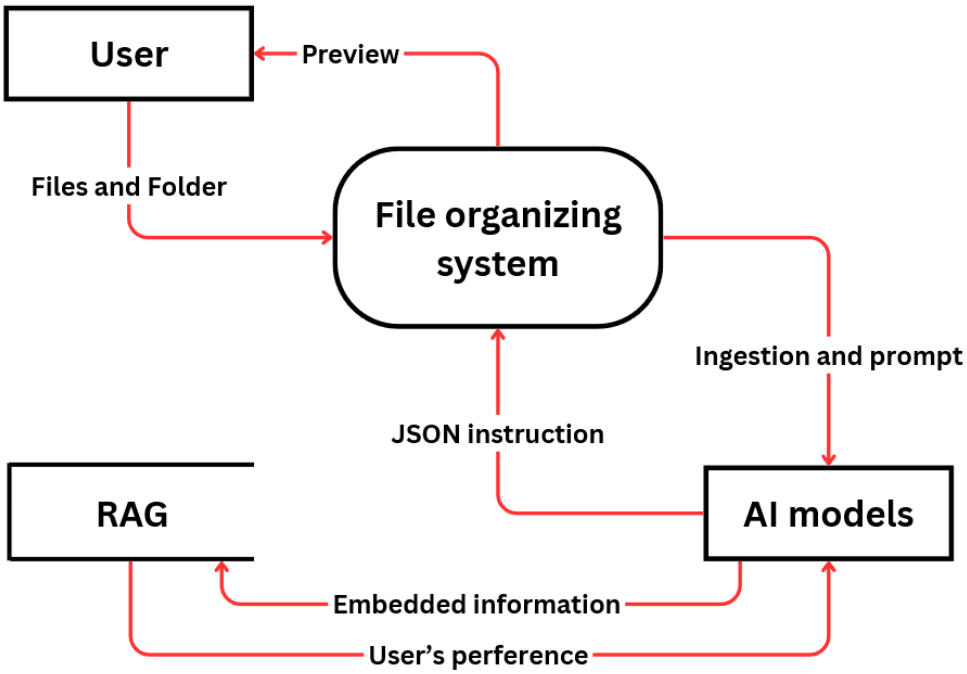


Figure 3.1: Dataflow Diagram (level 0)

3.2.2 Main sequence Diagram

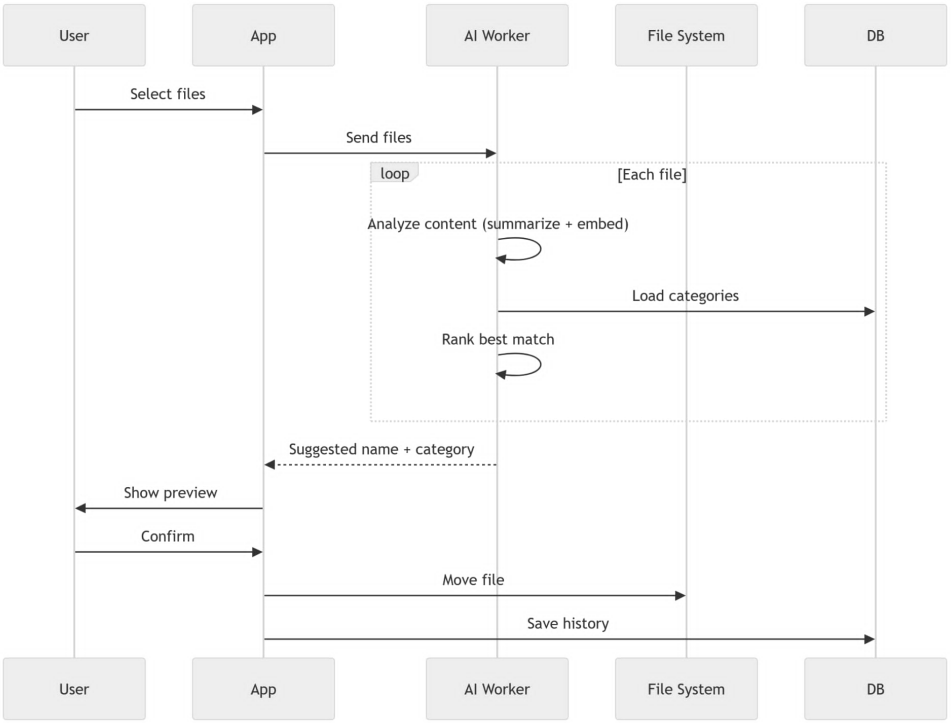


Figure 3.2: Main Sequence Diagram

3.2.3 Frontend

The project is built with Tauri + React. The frontend provides a desktop UI for analyzing dropped files, suggesting categories and names, tracking organization history, and managing settings through a single cohesive interface.

Core development

- File Analysis and Intake: Drag-and-drop and file picker mechanisms for capturing files into the organization pipeline
- Interface for AI models: LLM Models configuration set up
- Historical Tracking: Comprehensive logging and searchable repository of organization activities
- Settings and Configuration: User-accessible controls for category management, folder configuration, and automation policies
- Authentication Integration: Google Calendar API access
- Background Automation: Real-time monitoring and queued file processing with concurrency control

Table 3.1: Tools and Technology implementation in frontend

Component	**Technology**	**Rationale**
UI Framework	React	Modern concurrent rendering with automatic batching
Language	TypeScript	Compile-time type safety and IDE support
Build Tool	Vite	Rapid build times and optimized production bundles
Routing	React Router	Client-side navigation with protected route patterns
State Management	Zustand	Minimalist API with built-in persistence middleware
Component Library	shadcn/ui	Accessible, customizable Radix UI components
Styling	Tailwind CSS	Utility-first CSS with CSS variable theme support
Icons	Lucide React	Lightweight SVG icon library
Desktop Runtime	Tauri	Native desktop integration with Rust backend
Date Handling	date-fns	Functional, modular date manipulation library
Data Visualization	recharts	React-based charting library for metrics display
Notifications	Sonner	Toast notification system for user feedback
Package Manager	Bun	Fast JavaScript runtime and package manager

Architecture Layer

The frontend architecture will follow a strict layered structure, where each component is responsible for exactly one concern. The processing pipeline is organized as follows:

UI → IPC (invoke) → IPC Handler → Command → Service → Infrastructure → Repository

This design is grounded in the Clean Architecture principle introduced by Robert C. Martin (2012), which states that software should be organized into concentric layers where outer layers depend on inner layers, never the reverse. Each layer in the system maps to a specific responsibility:

Table 3.2: System architecture layers and responsibilities

Layer	Component	Responsibility
UI	React/TypeScript	Render state and capture user intent
IPC	Tauri invoke()	Serialize and transmit commands across the process boundary
IPC Handler	Tauri command registration	Deserialize input, validate it, and route it to the appropriate command
Command	Rust command functions	Orchestrate one use case end-to-end
Service	Domain service classes	Encapsulate business logic
Infrastructure	File system, watcher, process manager	Execute side effects against the operating system
Repository	JSON / SQLite persistence	Read and write application state

In Tauri, the frontend and the backend (Rust) run in separate processes with no shared memory. JavaScript cannot directly access the file system, spawn processes, or interact with the operating system. This is by design, Tauri enforces a strict security boundary between the web context and native capabilities, similar to how web browsers sandbox JavaScript. All cross-boundary calls must pass through the IPC channel (invoke()), which Tauri validates against a declared capability manifest before executing.

This constraint naturally enforces the Separation of Concerns principle (Dijkstra, 1974). The UI layer contains no file-system logic; the repository layer contains no rendering logic. Each layer can be reasoned about, tested, and modified independently. If the storage format for categories changes from JSON to SQLite, only the repository layer changes the command, service, and UI layers are unaffected.

Furthermore, separating the Command layer from the Service layer allows the same business logic (e.g., moving a file and logging the event) to be reused across multiple entry points in a manual user action, an automated watcher trigger, or a future scheduled task without duplicating code.

Organization Design

The organization feature is the core workflow of this project. Its design addresses three fundamental challenges:

1. Files may arrive one at a time or in large batches.
2. AI analysis is slow, non-deterministic, and expensive per call.
3. File moves are irreversible; user confirmation is required before any destructive action.

The full pipeline is divided into four phases: File Discovery, Frontend Queue, AI Analysis, and User Confirmation and Execution.

Phase 1: File Discovery Files enter the organize pipeline through two mutually exclusive triggers: Trigger A: Manual Selection The user initiates a native file picker dialog through the UI. Which will cross from IPC boundary into the Rust backend. The Rust command opens the operating system's native dialog using the rfd (Rusty File Dialogs) library and returns the selected paths to the frontend. This approach is used because a native dialog operates with full OS-level permissions and provides a familiar, OS-consistent experience. The JavaScript layer has no direct access to the file system path namespace; Tauri grants it only the specific paths the user explicitly chose. Trigger B: Automatic Detection via File System Watcher When the user has configured a watched folder, the Tauri backend registers a file system watcher using the notify crate. When a

new file is created or modified in the watched directory, the watcher does not immediately forward the event. Instead, it will apply a two-stage validation:

- Debounce (500 ms quiet window): File system events fire multiple times during a single write operation. A debounce window collapses all events for the same path into one, preventing duplicate processing.
- Stability check (polling size and mtime for up to 30 retries): To prevent the system from attempting to analyze a file that is still being downloaded or written (e.g., .crdownload or .part files).

Once confirmed, Tauri emits a watched-file-added event to the frontend via the Tauri event system. This design choice keeps the Rust layer decoupled from the AI pipeline and lets the frontend apply the same queue-based flow regardless of how the file was discovered.

Phase 2: Frontend Queue Architecture When files arrive (either from manual selection or from watcher events), the frontend does not immediately dispatch all files to the AI worker in parallel. Instead, it inserts each file path into a sequential analysis queue managed in the Zustand store. Why is queues necessary: This decision is justified by the Queue-Based Load Leveling pattern (Microsoft Azure Architecture Patterns, 2023), which states that a queue should be placed between a producer (the UI) and a slow consumer (the AI process) to prevent the consumer from being overwhelmed during load spikes. Specifically:

1. The AI pipeline is not instantaneous. Each file requires LLM inference for summarization, embedding computation, and cosine similarity scoring against all categories. Processing multiple files in parallel would cause resource contention on the llama-server, increasing total latency per file rather than reducing it.
2. Ordered processing is predictable. A first-in-first-out (FIFO) queue ensures files are processed in the order they were discovered, which matches user expectations when dropping a batch of files.
3. Progress visibility. Because files are in a known queue, the UI can render a progress indicator (”3 of 7 files analyzed”) accurately. Without a queue, concurrent fire-and-forget dispatches make it impossible to know the current processing state.

Phase 3: AI Analysis For each file dequeued, the frontend dispatches an HTTP POST request to backend service, which is a separate Python process (FastAPI) that runs alongside the Tauri app as a sidecar. Separating the AI pipeline into its own process rather than embedding it in Rust or JavaScript is a deliberate architectural choice:

- Language ecosystem: Python has matured, well-maintained libraries for AI tasks implementing these in Rust would require significant effort with less library support.
- Process isolation: If the AI worker crashes due to an out-of-memory error or model failure, it does not crash the UI process. The Tauri backend can detect the failure and restart the sidecar independently.
- Independent scalability: In future versions, the worker could be moved to a remote server without changing the frontend or Tauri code, since they only communicate over HTTP.

Phase 4: User Confirmation and File Execution After the analysis result is returned to the frontend, it presents the user with:

- A ranked list of category suggestions with confidence scores.
- Three alternative filename suggestions generated by LLM.

- The original file path for reference.

Why user confirmation is mandatory before any file move: File moves are destructive and irreversible under normal circumstances (the original path no longer exists) so it requires that users should be able to undo actions or confirm before they are committed. Upon confirmation, the frontend calls invoke the method to move file. The Tauri Rust backend executes the actual file system operation. Using a Tauri command for the file move, rather than having Python backend process perform it, ensures that the file move is executed by the Rust process that holds the appropriate file system permissions declared in the Tauri capability manifest. After the move, the frontend writes history sessions to record the original and new paths in the local history store, enabling the undo operation. The undo flow simply reverses the action to move file followed by writing history record event with event type: ”undone”.

3.2.4 Backend

This project is a privacy-first local file organization system that assists users in categorizing and organizing files based on semantic understanding of file content. The system operates as a desktop application, processing files from absolute paths without uploading or transmitting content outside the local system boundary.

Core development

- Scanner Service: Enumerates files from specified directory paths and extracts file metadata: path, size, content hash (SHA-256), extension
- Classification: Computes cosine similarity between file embeddings and category embeddings
- RAG Service: A retrieval-augmented generation. Manages the embedding function pipeline, delegating to llama-server. Supporting semantic search
- Parser: Extracts structured text and tables from PDF, DOCX, XLSX, PPTX, HTML, and Markdown files. Supports two profiles (”fast” and ”rich”) with configurable for normal OCR or table structure extraction
- LLM Client: Async HTTP interface to out-of-process llama-server. Essential method for the access of AI. Supporter of the features such as files organization, summary
- Database: Cache on activity. Participating in workflow for the purpose of reducing workload and remembering the already processed file

Table 3.3: Tools and technology implementation in backend

Category	**Tool**	**Purpose**
Framework	FastAPI	Web API framework
Database	SQLAlchemy + SQLModel	Object-relational mapping
Migrations	Alembic	Schema versioning
Parsing	Docling	Multi-format document parsing
RAG	RAG-Anything	Semantic indexing
RAG Backend	LightRAG	Vector storage
LLM	llama.cpp	Local LLM inference as llama-server
Vectors	NumPy	Numerical operations

Logging	logging	Structured logging
Config	dataclass + env	Settings management

AI models implementation

The system must be able to read and support document files, including .pdf, .docx, .pptx, .xlsx and image files. Some file types are easily understood by LLM because they were trained by them, as text base documents, but some like image files (png or .jpeg) cannot be comprehended by normal LLM, and that is the reason why VLM is needed.

With the integration of RAG system and vector database to support semantic search, LLM also need to embedded word from file's content into the vector database which may interfere with its normal process and increasing more time on each session of activity. One of the solutions is to use separate embedding models to handle this job.

This project prioritizes working in local environments, that means special interface is needed. We chose llama.ccp because it requires minimal setup, suitable for offline desktop application.

In summary of this part

- All models will be of .gguf format for the use with llama.cpp
- There will be VLM + mmproj (image encoder support) and embedding model for the best practice

Workflow

The analysis pipeline inside the application applies a cache-first strategy:

- **Scan**: The file hash, size, and extension are computed.
- **Cache check**: A categories hash is computed from the current set of active categories and the parser profile. If the analysis record already exists in the database for this file with a matching hash, the cached analysis is returned immediately and no LLM call is made.
- **Parse**: If the cache misses, the file is parsed using Docling's fast profile to extract text content. The result is a structured summary suitable for LLM input.
- **Summarize**: The extracted text is sent to the llama-server chat endpoint with a summarization prompt. For image files, the vision-capable model endpoint is used instead.
- **Rename suggestion**: The summary is sent to the LLM with a rename prompt that instructs the model to produce filenames. The output is post-processed with sanitization rules to ensure filesystem compatibility.
- **Embedding and classification**: The file summary is embedded using the llama-server embedding endpoint. The resulting vector is compared against the pre-computed embedding of each active category using cosine similarity.
- **Persist**: The analysis result is written to the database and a history entry is created.

So, the basic workflow should be like this:

[Stage 1] Scan Files

- Enumerate file paths

- Compute file hashes and metadata
- Load existing File records from database
- Determine which files have changed

↓

[Stage 2] Cache Check

- If unchanged & categories match → Hit (return cached results)
- If categories changed but file unchanged → Partial (re-classify only)
- If file changed or forced → Full pipeline

↓

[Stage 3a: Full pipeline] Parse & Queue Background Ingestion

- Parsing and extracting
- Enqueue file into background RAG ingestion

↓

[Stage 3b] Generate Summary (if not cached)

- If image: send to VLM
- If text: use extracted text + LLM summary
- If neither: filename fallback

↓

[Stage 4] Generate Rename Suggestions (if summary available)

- LLM generates N candidate filenames based on summary
- Sanitize and validate suggestions

↓

[Stage 5] Classify Files

- Compute file embedding (filename + summary)
- Load active categories with embeddings
- Compute cosine similarity for each category
- Sort by score descending; retain top-K

↓

[Stage 6] Return Results

- Format response with scores, suggestions, metadata
- Log history activity

↓

[Response] (with scores, suggestions, timings)

3.3 Evaluation method

To conform with objectives of this project which is a service invention project in the form of a desktop application, the evaluation method must include both Performance testing and End-user opinion. To cover both aspects in a structured way, the evaluation strategy of this project is organized into three complementary layers, each answering a different question about the system:

- **Automated testing (code-level):** Unit tests written alongside the source code to verify the correctness of individual functions, modules, and persistence layers. The frontend is tested with the Bun test runner (`bun test`), with test files organized in a dedicated `tests/` directory mirroring the `src/` hierarchy. The backend is tested with `cargo test --lib`, with tests organized in a dedicated `src/tests/` module tree mirroring the source hierarchy and gated by a single `#[cfg(test)] mod tests;` declaration in `lib.rs`. This layer protects against regressions during development and enforces the data contract between the frontend and backend.
- **Persona-based benchmark (system-level):** An automated functional benchmark that measures the application's file-classification accuracy on realistic test files grouped by user persona (Student, HR Officer, Designer). This layer answers whether the system behaves correctly on real-world inputs.
- **Expert evaluation form (user-level):** A questionnaire-based evaluation completed by experts in computer engineering or software development. The questions are derived from the project objectives, and an odd number of experts participate so that a consensus on the application's success can be obtained.

Among the three layers, the Expert Evaluation form is treated as the primary qualitative method for the overall performance analysis and conclusion of this project, while the automated tests and the persona-based benchmark provide the supporting quantitative evidence.

CHAPTER 4 RESULTS AND ANALYSIS

4.1 Finished Product

The project "AI-powered file organizer application", named KLIN, was conducted according to the following steps:

- Studying project-related information
- Developing a demo application and testing the LLM model
- Creating an expert evaluation form for application performance
- Collecting data for development and improvement
- Analyzing data and summarizing the project results

The remainder of this section presents the finished application through screenshots of the actual product, grouped by feature area.

First-Time Setup

When KLIN is launched for the first time, it guides the user through a short setup process. The user is welcomed by the setup screen, the required local AI models are downloaded, and a four-step wizard collects the base path, categories, and watched folders. All AI models run entirely on the user's device, so files and notes never leave the local machine.

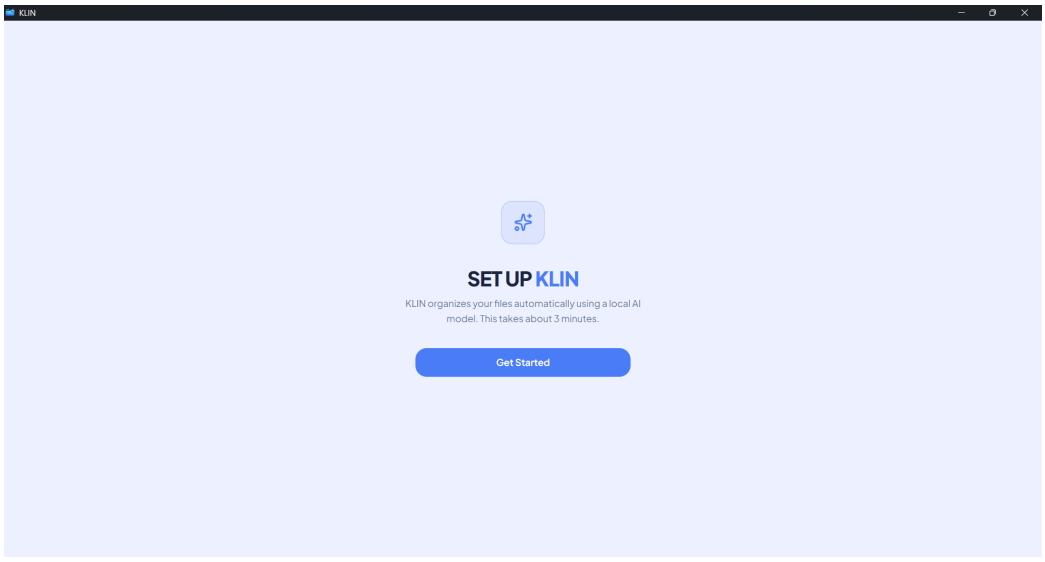


Figure 4.1: The first-time setup welcome screen

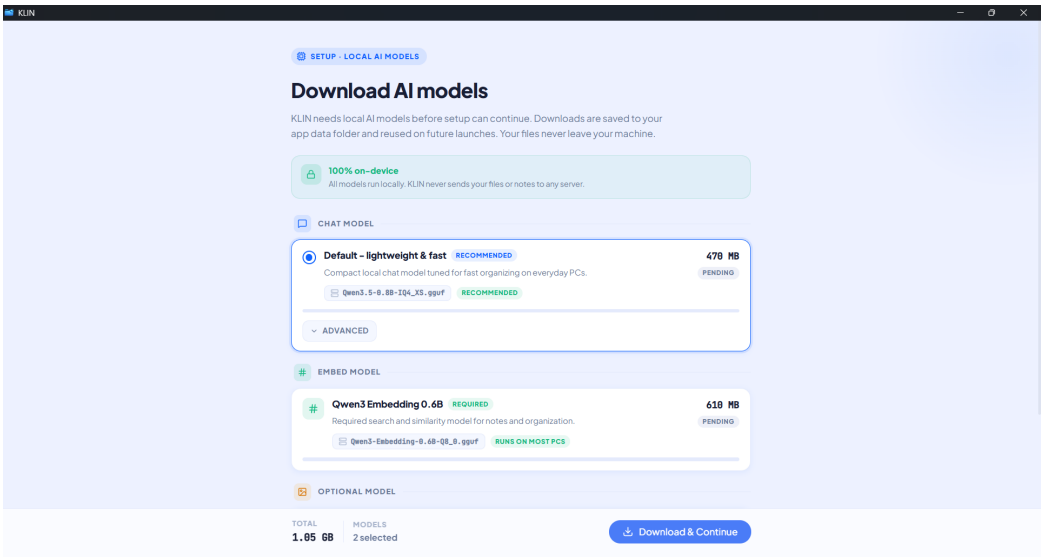


Figure 4.2: Downloading the local AI models during setup; all models run on-device

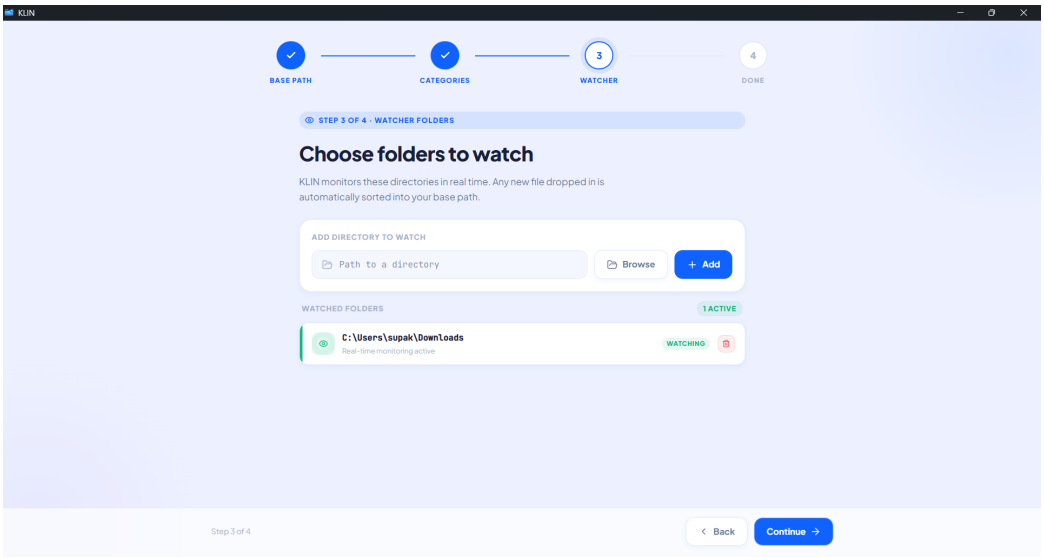


Figure 4.3: The guided four-step setup wizard (base path, categories, watched folders, and confirmation)

Dashboard

The Dashboard is the main screen of the application. It summarizes the organized categories and their file counts, provides the AI Organizer drop zone for adding new files, and displays a live activity feed of recent file movements.

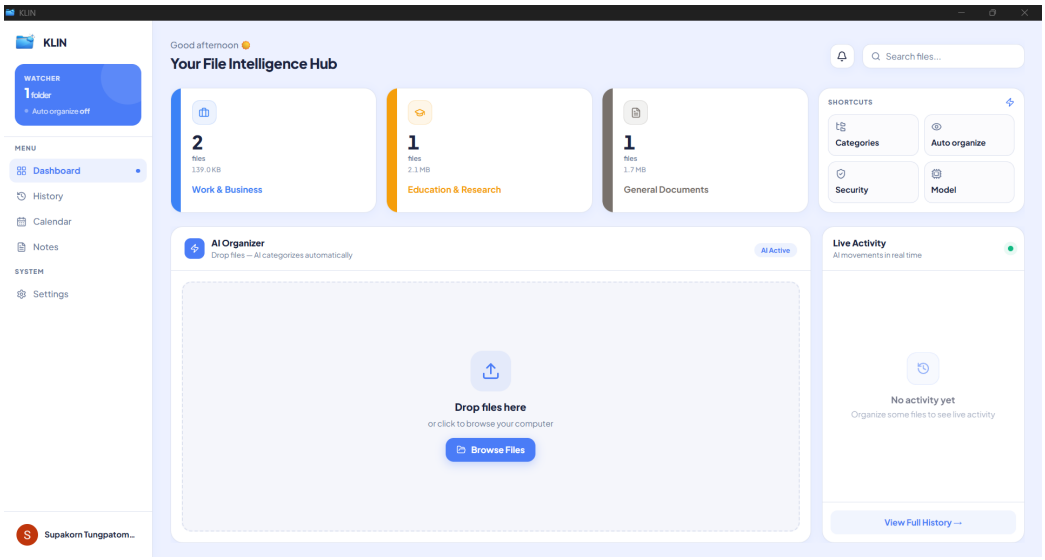


Figure 4.4: The Dashboard (File Intelligence Hub) of the application

AI File Organizer

The AI File Organizer is the core feature of the application. After files are added, the system analyzes each file's content and presents the top five category suggestions together with the proposed destination folder. The user reviews these suggestions in a preview before approving the move.

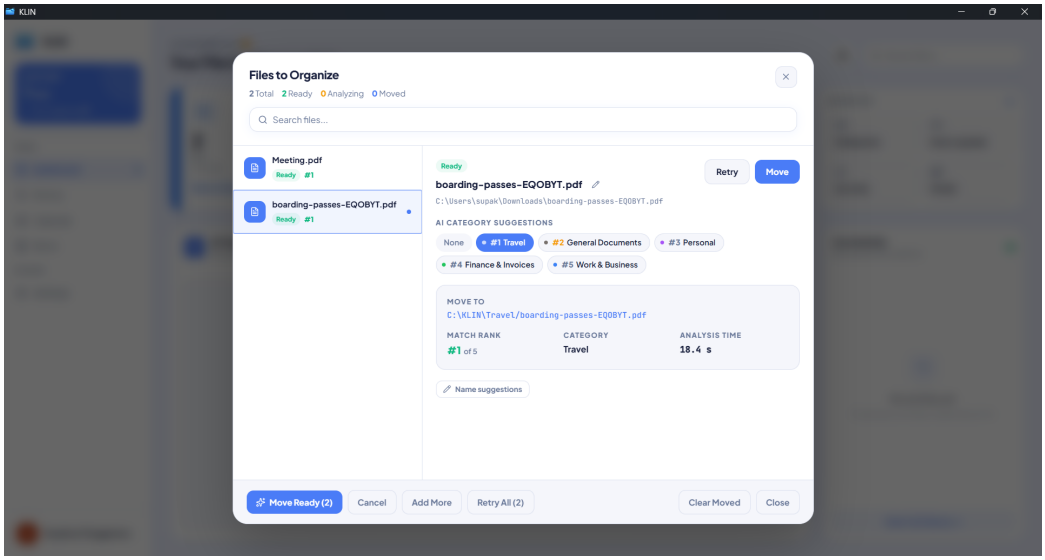


Figure 4.5: The AI Organizer showing the top-five category suggestions and the proposed destination for a file

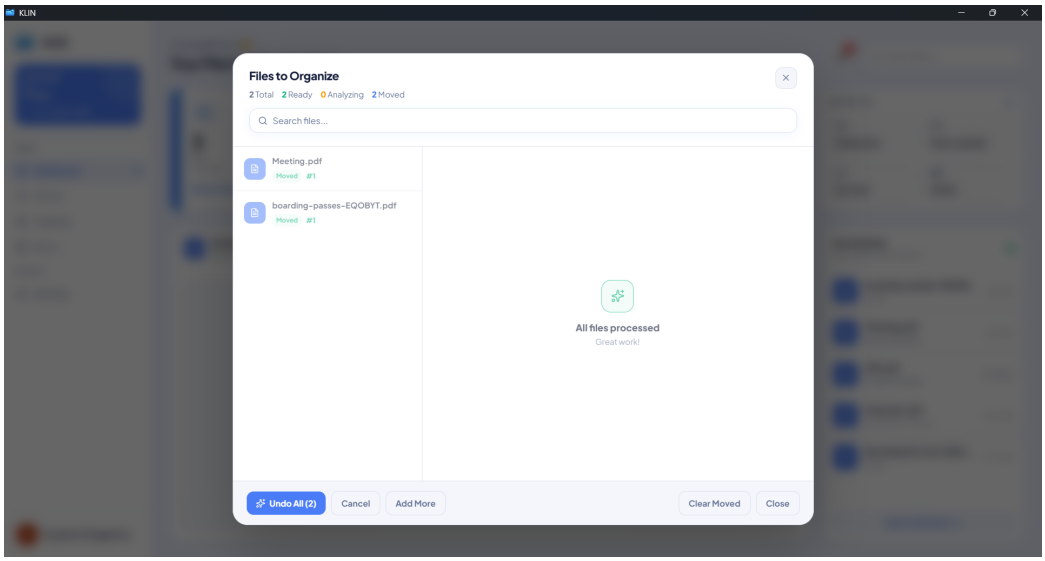


Figure 4.6: The AI Organizer after all files have been processed and moved

Activity History

Every file operation performed by the application is recorded in the Activity History page. Each entry can be reverted individually, and the page also supports undoing or redoing all recorded operations.

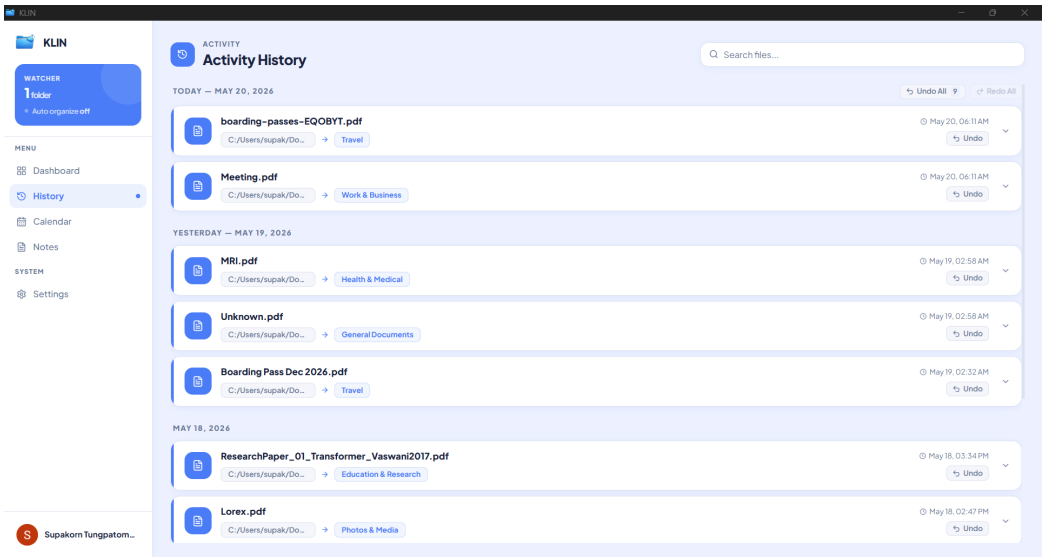


Figure 4.7: The Activity History page, listing every file operation with per-item Undo

Calendar Integration

The application can detect calendar events contained in documents, such as meeting or interview dates, and add them to the user's Google Calendar. The Calendar page displays these events and stays synchronized with Google Calendar.

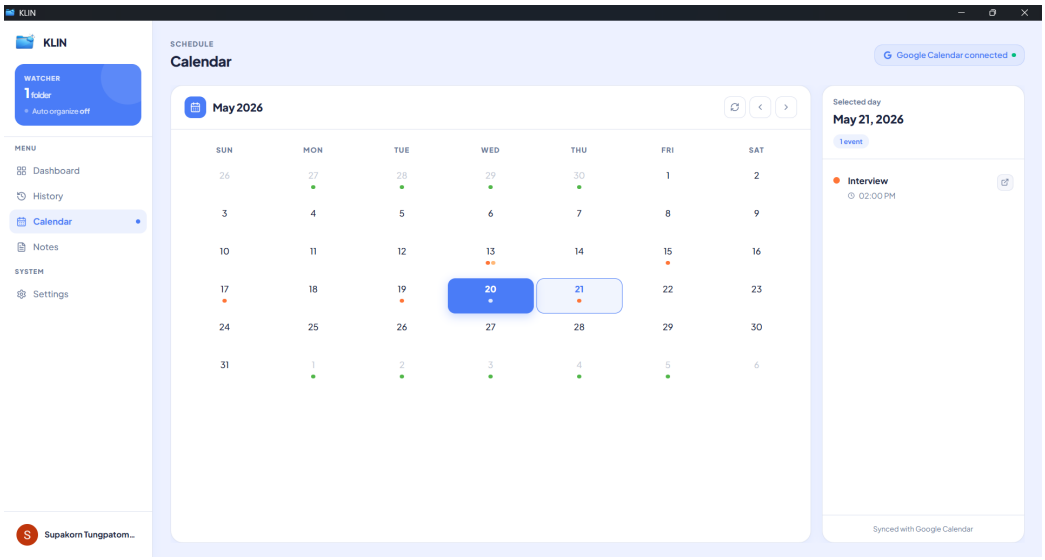


Figure 4.8: The Calendar page, synchronized with Google Calendar

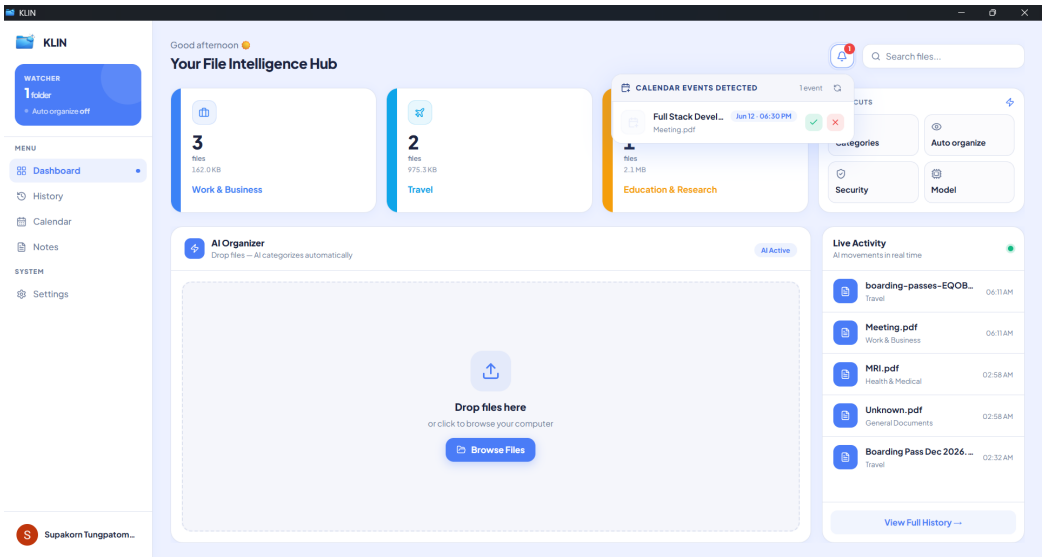


Figure 4.9: Automatic detection of a calendar event extracted from a document

Notes and AI Summaries

The Notes page lets users keep notes alongside their files. Using the AI Summarize function, the application can read a document and generate a concise summary note, allowing the user to understand a file's content without opening it.

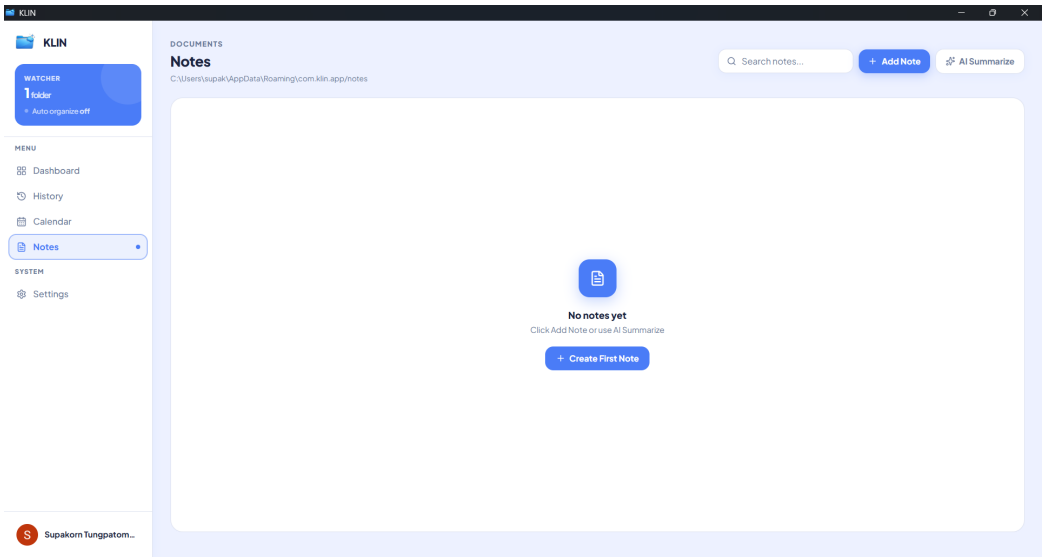


Figure 4.10: The Notes page of the application

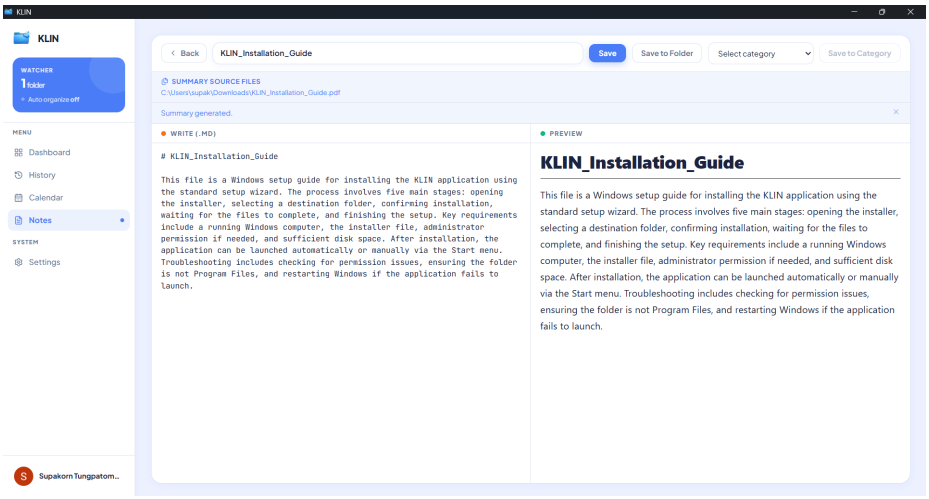


Figure 4.11: An AI-generated summary note created from a document

Application Settings

The Settings page is divided into five sections. The Account section manages the connected Google account; Configuration sets the default output folder, watched folders, and AI categories, and can also import an existing folder structure as categories in a single batch operation; Automation controls the Auto Organize background watcher and manual scanning; Model manages the local AI models; and Security provides a vault for locking files and folders so that they are never sent to the AI.

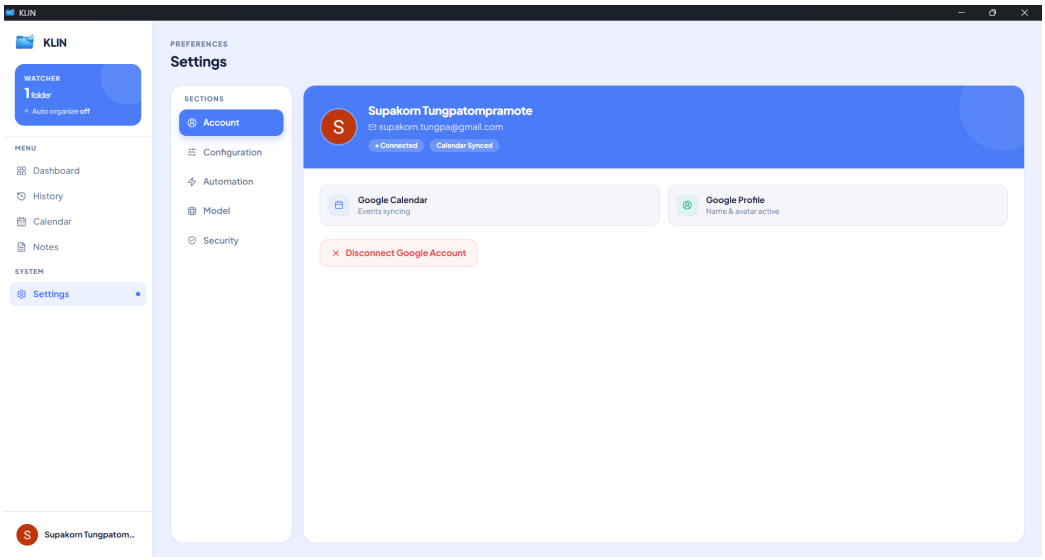


Figure 4.12: Settings – Account: the connected Google account

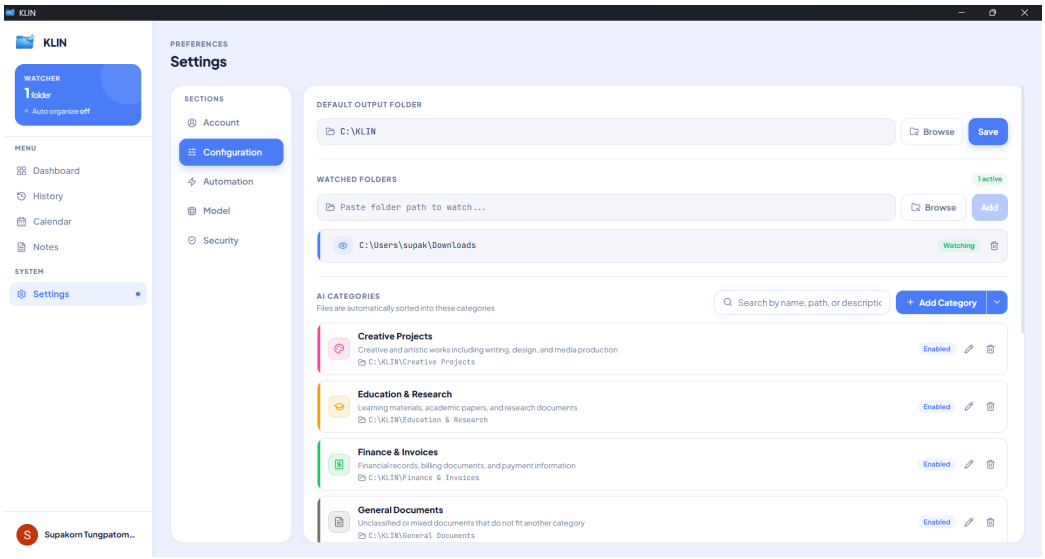


Figure 4.13: Settings – Configuration: default output folder, watched folders, and AI categories

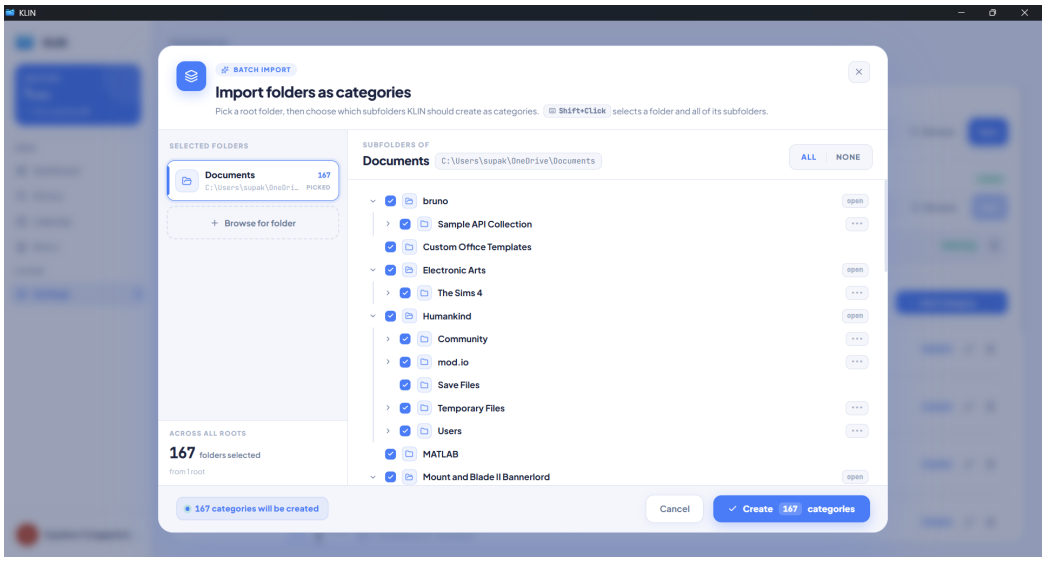


Figure 4.14: The batch import dialog, opened from the Configuration section, for importing an existing folder structure as AI categories in a single operation

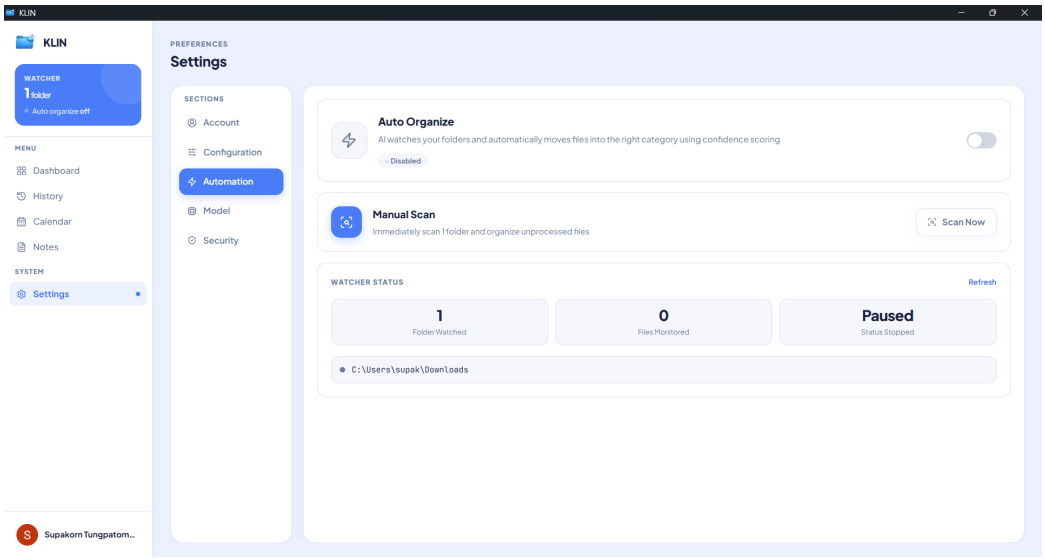


Figure 4.15: Settings – Automation: the Auto Organize watcher and manual scan

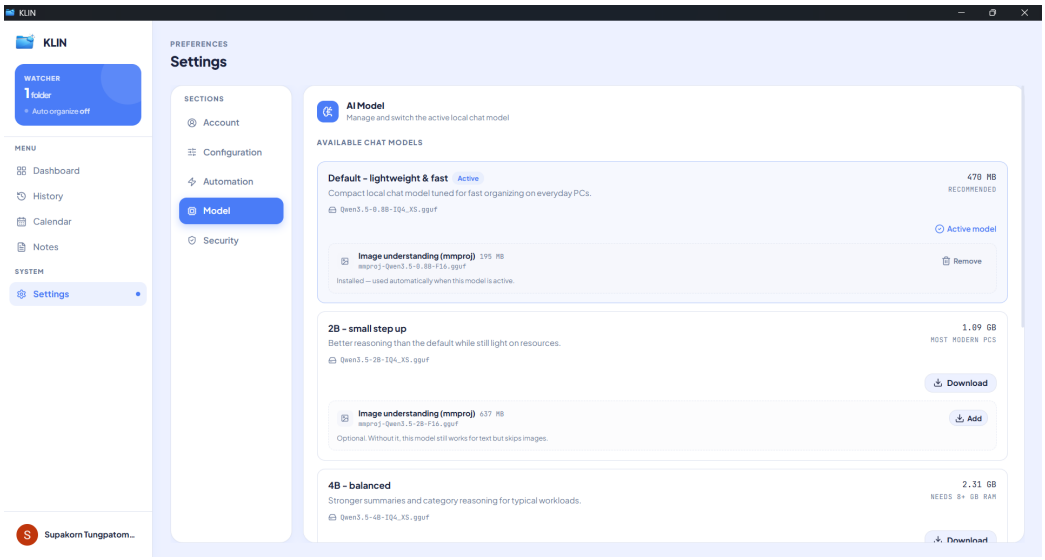


Figure 4.16: Settings – Model: management of the local AI models

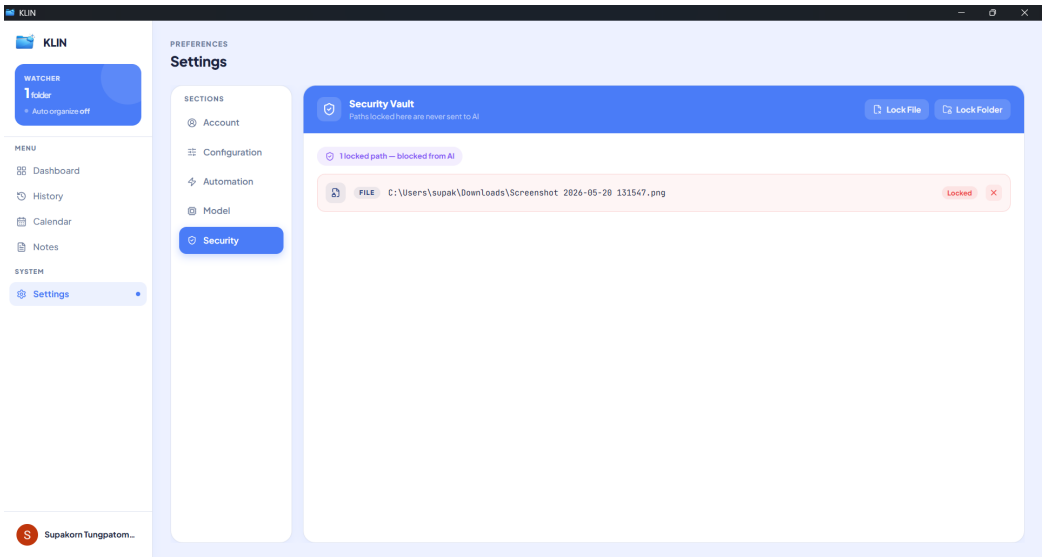


Figure 4.17: Settings – Security: the Security Vault for locking files and folders from AI processing

4.2 Automated Test Results

In addition to the higher-level evaluations described in the following sections, an automated unit-test suite was developed alongside the source code of both the frontend and the backend. The purpose of these tests is to verify code-level correctness, to enforce the data contract shared between the two layers (for example, the serde `camelCase` field naming used by all entities crossing the Tauri boundary), and to provide regression safety during ongoing development. All tests in both suites currently pass.

4.2.1 Frontend Test Suite

The frontend test suite is executed using `bun test`, Bun's built-in Jest-compatible test runner. Test files are organized in a dedicated `tests/` directory at the package root, whose internal structure mirrors the `src/` hierarchy (for example, tests for `src/lib/*` live under `tests/lib/*`, and tests for `src/stores/*` live

under `tests/stores/*`), and follow the `*.test.ts` naming convention. Imports use the `@/*` TypeScript path alias so each test references its module under test by the same logical path as the rest of the application code. The scope of the suite covers pure utility functions and the Zustand state stores; React components, Tauri `invoke` calls, and end-to-end flows are intentionally not covered at this layer, as they are exercised by the persona-based benchmark and the expert evaluation. The suite contains **72 tests across 8 files** (91 assertions), all passing, with a total runtime of approximately 545 ms.

Table 4.1 Frontend unit-test files and their coverage

Module under test	**What it covers**
Category helpers	Category name normalization, matching, and defaults
Error helpers	Error-message extraction and error-shape normalization
Path helpers	Path joining, parent extraction, and separator normalization across Windows and POSIX
Scoring helpers	Score ranking, top-category selection, and tie handling
Text helpers	String formatting, truncation, and sanitization
General utilities	Class-name merging and other shared helpers
Rule store (Zustand)	Rule create, read, update, delete actions and store-state transitions
Undo/redo store	Undo and redo stack, history bounds, and state restoration

4.2.2 Backend Test Suite

The backend test suite is executed using `cargo test --lib`. Tests are organized in a dedicated `src/tests/` module tree whose internal structure mirrors the source hierarchy: each production module (for example `services/rule_service.rs`) has a corresponding test file at the equivalent path under `src/tests/` (in this example `src/tests/services/rule_service_tests.rs`). The entire test tree is gated by `#[cfg(test)] mod tests;` declared in `lib.rs`, which guarantees that all test code – together with the `tempfile` development dependency used for filesystem fixtures and the hand-rolled in-memory fake repositories used for service-layer tests – is stripped from release builds and has zero runtime impact on the production binary. The scope covers domain serde contracts, JSON-backed repositories (exercised against temporary directories), service-layer orchestration (exercised against in-memory fake repositories), and the sidecar idle-stop timer. Tauri command handlers, filesystem watchers, the sidecar subprocess manager, OAuth, and application paths are intentionally not covered, as they are runtime-coupled and are validated through the higher-level evaluations. The suite contains **37 tests across 9 modules**, all passing, with a total runtime of approximately 30 ms.

Table 4.2 Backend unit-test modules and their coverage

Module under test	Layer	What it covers
Domain entities	Domain	Domain-entity serialization contract and JSON round-trip encoding and decoding (prevents accidental field renames from breaking the frontend)
Scoring strategy	Domain	Scoring-provider identity and factory selection
Rule repository	Persistence	Rule persistence: missing, empty, and malformed file handling, save/list round-trip, and overwrite behavior
Category repository	Persistence	Category persistence: default fallback when no file exists, JSON parsing, and malformed-input error handling
History repository	Persistence	History-log persistence: append and list ordering, empty-file handling, and parse-error reporting
Automation-config repository	Persistence	Automation-config persistence: default fallback, save/load round-trip, and automatic parent-directory creation
Rule service	Service	Rule service orchestration: delegation to the repository and propagation of list and save errors
Category service	Service	Category service orchestration: pass-through to the repository, empty-result handling, and error propagation
History service	Service	History service orchestration: write and list ordering, and error propagation
Llama sidecar	Sidecar	Sidecar idle-stop timer: triggers after the timeout, holds before the timeout, and stays disabled when no activity has been recorded

4.2.3 Worker Test Suite

A dedicated test suite for the background worker layer is planned for a future iteration of the project. This suite will cover the worker's job-scheduling logic, the IPC contract between the worker and the main backend process, and error-recovery behavior when worker tasks fail or are cancelled. Once implemented, its results will be reported here following the same structure used for the frontend and backend suites above, and an additional column will be added to the combined summary table in Section 4.2.4.

4.2.4 Combined Test Summary

Across both currently implemented suites, the project has **109 automated tests in total** (72 frontend + 37 backend), all passing, with a combined runtime of approximately 575 ms. The summary below consolidates the runners, conventions, and scope of each suite; the Worker column is reserved for the planned worker-layer suite described in Section 4.2.3.

Table 4.3 Combined summary of the automated test suites

	Frontend	**Backend**	**Worker (planned)**
Runner	`bun test`	`cargo test --lib`	—
Test files	8	9	—
Tests	72	37	—
Result	72 pass / 0 fail	37 pass / 0 fail	—
Duration	∼545 ms	∼30 ms	—
Style	Separate `tests/` directory at the package root, mirroring the `src/` hierarchy; imports use the `@/*` path alias	Separate `src/tests/` module tree mirroring the source hierarchy, gated by `#[cfg(test)] mod tests;` in `lib.rs`	—
Scope	Pure utils + Zustand stores	Domain serde, JSON repos (with `tempfile`), services (with fake repos), sidecar idle-stop	—
Out of scope	React components, Tauri `invoke`, E2E	Tauri command handlers, fs watchers, sidecar subprocess, OAuth	—

4.2.5 Discussion

The automated test suites provide strong confidence in the code-level correctness of the project: the 72 frontend tests verify that the pure utility functions and Zustand stores behave as specified, and the 37 backend tests verify that the domain entities serialize correctly across the Tauri boundary, that the JSON-backed repositories handle missing, empty, and malformed inputs safely, and that the service layer correctly delegates to and propagates errors from its repositories. The serde round-trip tests on the domain entities are particularly important, as they prevent accidental field renames from silently breaking the frontend–backend contract.

These tests, however, do not by themselves prove that the application is useful or that the end-to-end user experience is correct. Component-level rendering, the live Tauri `invoke` bridge, filesystem watchers, the sidecar subprocess, and the AI classification quality are deliberately validated by the higher-level evaluations: the persona-based benchmark in Section 4.3 measures classification accuracy on realistic inputs, and the expert evaluation in Section 4.4 captures qualitative judgements about usability, security, and overall fit for real-world use. Taken together, the three layers form a complete evaluation of the project from code level up to user level.

4.3 Persona-Based Benchmark Evaluation

In addition to the expert evaluation, an automated benchmark was conducted to quantitatively measure the file classification accuracy of the "AI-powered file organizer application" across different real-world user profiles. This benchmark complements the expert evaluation by providing objective, reproducible measurements of the system's core classification capability.

Benchmark Setup

Three user personas were defined to represent distinct document domains commonly encountered in real-world use:

- **Persona 1 – Student:** test files drawn from six folders representing typical academic work (Assignments, ExamPrep, LabReports, LectureSlides, ProjectFiles, and ResearchPapers).
- **Persona 2 – HR Officer:** test files drawn from five folders representing common human-resources documents (EmployeeRecords, MeetingMinutes, PoliciesAndProcedures, Recruitment, and TrainingMaterials).

- **Persona 3 – Designer:** test files drawn from six folders representing creative-industry documents (ClientBriefs, ContractsAndProposals, InspirationAndReferences, Invoices, Portfolio, and ProjectDeliverables).

For each persona, 20 test files were prepared (60 files in total). Each file was passed to the application, which returned its top-5 ranked folder suggestions. The result for each file was then classified using the following scoring criteria:

- **Correct** – the top-1 suggestion matches the correct destination folder.
- **Almost Correct** – the top-1 suggestion is incorrect, but the correct folder appears as the second-ranked suggestion.
- **Incorrect** – the correct folder appears at rank 3–5, or does not appear in the top-5 suggestions at all.

Benchmark Results

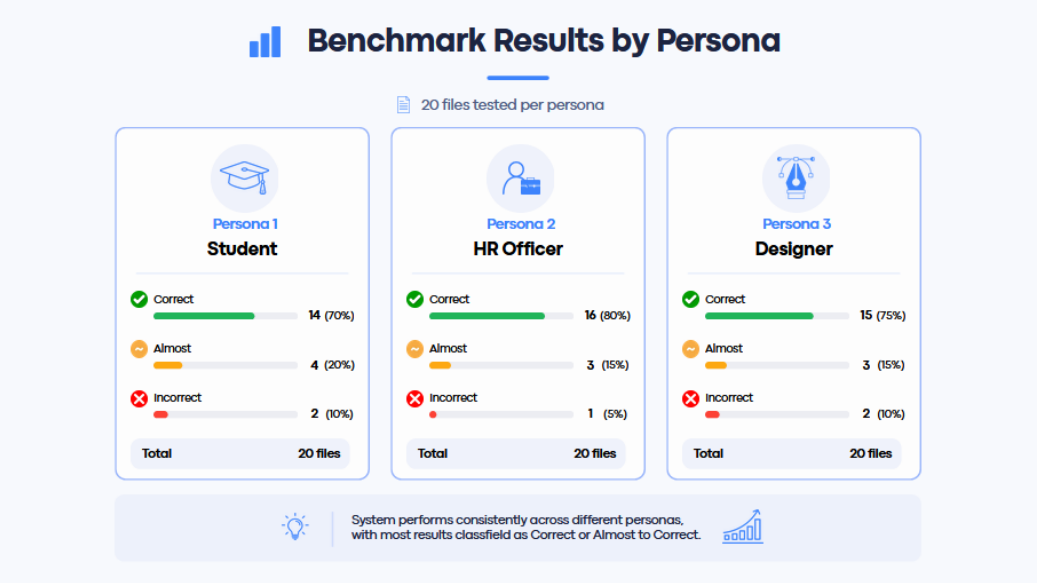


Figure 4.18: Benchmark results by persona (20 files tested per persona)

Table 4.4 Summary of benchmark results across the three personas

Persona	Correct	Almost Correct	Incorrect	Total
Persona 1 – Student	14 (70%)	4 (20%)	2 (10%)	20
Persona 2 – HR Officer	16 (80%)	3 (15%)	1 (5%)	20
Persona 3 – Designer	15 (75%)	3 (15%)	2 (10%)	20
Overall	**45 (75.0%)**	**10 (16.7%)**	**5 (8.3%)**	**60**

Analysis

The benchmark results show that the application performs consistently across all three personas. Considering the top-1 suggestion alone, the system achieved an average accuracy of 75.0%, with the HR Officer persona producing the highest accuracy (80%) and the Student persona the lowest (70%). When the "Almost Correct" cases are included – that is, when the correct folder appears within the top-2 suggestions – the combined

acceptable rate rises to 91.7% overall (90% for Student, 95% for HR Officer, and 90% for Designer). Only 8.3% of files (5 of 60) fell into the Incorrect category.

These results indicate that the system generalizes well across document domains with very different vocabulary and structure, from academic papers to HR policy documents to creative client briefs. The slightly lower accuracy on the Student persona can be attributed to overlapping content between related academic folders (for example, lab reports and project files often share similar terminology), while HR documents tend to use more distinctive domain-specific vocabulary that the model classifies more reliably. Overall, the benchmark supports the expert evaluation findings and confirms that the application's classification capability is suitable for practical, real-world use across diverse user profiles.

4.4 Methodology

Expert Evaluation tool for Application Performance

The Evaluation tool used to evaluate the application was a performance evaluation form, which will be filled by 3 experts from the field of computer engineering or software development, can be divided into two sections:

Part 1: Performance evaluation of the "AI-powered file organizer application" by experts. The project developers defined a 5-point scoring scale with the following meanings:

1. : unable to solve the problem.
2. : able to solve the problem partially, but some gaps remain.
3. : able to solve the problem appropriately, but further improvement is still possible
4. : able to solve the problem effectively with minor improvements possible.
5. : able to solve the problem completely and appropriately.

Part 2: Additional comments and suggestions on the performance evaluation of the "AI-powered file organizer application"

The data analysis in this project was conducted using computer software packages. The statistics used for analysis were as follows:

Data from the performance evaluation form of the "AI-powered file organizer application" were analyzed using the mean (X) and standard deviation (S.D.). The interpretation of the mean scores was defined as follows (Boonchom Srisa-ard, 2002: 162):

Mean score between 4.51 – 5.00 means able to solve the problem completely and appropriately.
Mean score between 3.51 – 4.50 means able to solve the problem effectively with minor improvements possible.
Mean score between 2.51 – 3.50 means able to solve the problem appropriately, but further improvement is still possible.
Mean score between 1.51 – 2.50 means able to solve the problem partially, but some gaps remain.
Mean score between 1.00 – 1.50 means unable to solve the problem.

Data Analysis

The project developers analyzed data from the expert performance evaluation form for the “AI-powered file organizer application”. The evaluation was conducted by 3 experts. The analysis results were presented in tables together with descriptive explanations and divided into two parts as follows:

Part 1: Analysis results of the expert performance evaluation of the “AI-powered file organizer application”. The analysis used the mean and standard deviation, both overall and by individual evaluation items. To make the data analysis easier to understand, the developer defined the statistical symbols and abbreviations used in the study as follows:

n refers to the sample size.

X refers to the mean score of satisfaction.

(S.D.) refers to the standard deviation.

Part 2: Additional comments and suggestions on the performance evaluation of the “AI-powered file organizer application”.

Data Analysis Results

Part 1: Analysis results of the expert performance evaluation of the “AI-powered file organizer application” using the mean and standard deviation, both overall and by individual evaluation items.

Table 4.5: Mean, standard deviation, and analysis of expert opinions regarding the performance of the “AI-powered file organizer application”

Evaluation Items	**Result**		
	X	**SD**	**Interpretation**
1. The system can accurately read and analyze data from PDF, Word, Excel, PowerPoint, TXT, and image files.	4.33	0.57	(4)
2. The system can accurately and appropriately categorize files based on their content.	3.66	0.47	(4)
3. The system can accurately and appropriately move files to destination folders according to defined templates.	5	0	(5)
4. The Preview system effectively helps reduce errors before moving or modifying files.	5	0	(5)
5. The history recording and Undo/Redo system is appropriate for practical use.	4	0	(4)
6. The UI design is easy to learn and use.	3.51	0.47	(4)
7. AI-related features, such as content summarization, note generation, and search are appropriate for practical use.	4	0	(4)
8. The system provides appropriate measures to prevent risks to user data.	5	0	(5)
9. The overall system architecture is appropriate for development as a desktop application.	4	0	(4)
10. The system has the potential to support real-world use on Windows.	4.66	0.47	(5)
Total	**4.31**	**0.19**	**(4)**

From Table 4.1, it was found that the experts’ opinions regarding the performance of the “AI-powered file organizer application” were overall at the level of able to solve the problem effectively with minor improvements

possible, with a mean score of 4.31. The standard deviation was 0.19, indicating a low level of dispersion from the mean. This means that the experts' opinions regarding the application's performance were similar and consistent. And when considering each evaluation item individually, four items were rated at the level of able to solve the problem completely and appropriately. These items were: The system can accurately and appropriately move files to destination folders according to defined templates, The Preview system effectively helps reduce errors before moving or modifying files, The system provides appropriate measures to prevent risks to user data, The system has the potential to support real-world use on Windows. The remaining evaluation items were rated at the level of able to solve the problem effectively with minor improvements possible.

Part 2: Additional Comments and Suggestions on the Performance Evaluation of the "AI-powered file organizer application"

Expert 1 evaluated that this application is of very good quality. This reflects a clear understanding of applying a Local LLM to solve real-world problems. The application has several strengths, including:

Strengths

- Use of Offline AI (llama.ccp) – This is an important strength, especially for organizations that require privacy and do not want to rely on cloud services.
- Preview & Undo System – This is very well designed and helps reduce the risk of incorrect file organization. It also provides good user experience.
- Full Thai Language Support – The system supports Thai documents, showing attention to the local context.
- Hybrid Approach (Keyword + AI) – Using keyword matching first and then sending the task to AI is a good concept. It helps save time and computational resources.

Areas for Improvement

- UI/UX – Although the application is usable, it is recommended to further study Material Design or Fluent Design, and Dark Mode may also be considered.
- Error Handling – Error handling should be clearer. For example, if AI models are not running or some process fails, the system should display easy-to-understand messages.
- Documentation – A complete user manual should be provided, including a troubleshooting guide.

Additional Suggestions

- Consider adding an adjustable Confidence Score Threshold for users, such as asking the user for confirmation if the AI confidence score is lower than 70%.
- Add a Batch Rename feature to rename files according to a predefined format.

Overall Comment

This application has academic value and can be applied in real-world use. It is recommended that the project be further developed as an Open-Source Project. The developers may also consider publishing a paper on the Hybrid Keyword-AI Approach for Thai-language documents.

Expert 2 evaluated that this “AI-powered file organizer application” is highly outstanding and useful, especially in educational and real-world working contexts.

Strengths

- Practical Usability – This is not only a sample project, but an application that can be immediately used to solve real problems in offices or government agencies, where large numbers of documents often need to be organized.
- Thai document parsing – This is a major strength because most government documents are in Thai and often include scanned documents that require OCR. The ability to support Thai language well makes the application practical for real use.
- 100% Offline Operation – This is very important for organizations that handle sensitive information, as they do not need to worry about data security risks from cloud services.
- Preview and Undo Features – Students often develop projects without considering these features, but this project clearly takes real users into account.

Application in Teaching

This project is a very good example for students to learn from in the following areas:

- Machine Learning Integration – It demonstrates integration with the AI models.
- File Handling – It involves managing multiple file formats.
- Real-World Problem Solving – It solves a real problem rather than simply following a basic example.

Areas for Improvement

- Create Tutorial Videos – Tutorial videos should be created to help general users understand how to use the application more quickly, especially users who are not familiar with technology.
- Improve the UI for Easier Use – Although the application is already usable, it should include a wizard or guided setup for first-time users.
- Add Use Case Examples – Preset templates should be provided, such as “organizing human resources documents” or “organizing financial documents.”

Additional Suggestions

- Consider adding multi-language support, starting with Thai and English, which has already been implemented, and possibly adding more languages in the future.
- Add Export/Import Settings so that users can share templates with their colleagues.
- Develop a Lite Version for low-specification computers, which may use only keyword matching without AI.

Overall Comment

This “AI-powered file organizer application” is highly suitable for presentation at an academic conference. It should also be considered for release as an Open-Source Project so that the community can benefit from it. This would also help build recognition for the institution and the developers.

Expert 3 evaluated this application from the perspective of real-world organizational use and data security.

Strengths

- Offline-first Architecture – Data does not leave the user’s device, making it suitable for organizations that handle sensitive information.
- Local AI Processing – This helps reduce the risk of data breaches.
- Password-protected File Support – The application takes data security into consideration.
- System Architecture – The code structure is well organized, and the installer is ready for use.

Areas for Improvement for Enterprise Deployment

- Security and Access Control – User authentication and role-based access control should be added for organizational use.
- Audit Trail – Logs should include more details, such as User ID, IP address, and file hash for integrity checking.
- Scalability – A database such as SQLite should be considered instead of JSON files when handling a large number of files.
- Network Deployment – A client-server architecture should be developed to allow multiple users to work together more conveniently.
- Backup and Recovery – Auto-backup mechanisms for configuration and export/import settings should be provided.

Additional Suggestions

- Consider Active Directory integration for organizations using a Windows Domain.
- Add API endpoints so that other systems can access and use the application.
- Develop a Health Check Dashboard for IT administrators to monitor system status.

Overall Comment

The application has a very strong foundation and is ready for personal use or small teams. However, if it is intended for use in large organizations, additional enterprise features should be developed according to the suggestions above. The use of Offline AI is an important strength in the on-premises solution market.

CHAPTER 5 CONCLUSION AND RECOMMENDATIONS

The project titled “AI-powered file organizer application” can be summarized, discussed, and presented with recommendations as follows:

5.1 Project Summary

Project Objectives

- To develop a desktop application that helps users manage files efficiently with AI.
- To reduce repetitive tasks and errors caused by manual file management.
- To improve the convenience of searching and organizing files through customizable file management templates.
- To establish a foundation for future expansion toward Agentic AI features, such as file content summarization, note generation, and calendar integration.

5.2 Methodology

Expert Evaluation tool for Application Performance

The Evaluation tool used to evaluate the application was a performance evaluation form, which will be filled by 3 experts from the field of computer engineering or software development, can be divided into two sections:

Part 1: Performance evaluation of the “AI-powered file organizer application” by experts. The project developers defined a 5-point scoring scale with the following meanings:

1. : unable to solve the problem.
2. : means able to solve the problem partially, but some gaps remain.
3. : means able to solve the problem appropriately, but further improvement is still possible
4. : means able to solve the problem effectively with minor improvements possible.
5. : means able to solve the problem completely and appropriately.

Part 2: Additional comments and suggestions on the performance evaluation of the “AI-powered file organizer application”

The data analysis in this project was conducted using computer software packages. The statistics used for analysis were as follows:

Data from the performance evaluation form of the “AI-powered file organizer application” were analyzed using the mean (X) and standard deviation (S.D.). The interpretation of the mean scores was defined as follows (Boonchom Srisa-ard, 2002: 162):

Mean score between 4.51 – 5.00 means able to solve the problem completely and appropriately.
Mean score between 3.51 – 4.50 means able to solve the problem effectively with minor improvements possible.
Mean score between 2.51 – 3.50 means able to solve the problem appropriately, but further improvement is still possible.

Mean score between 1.51 – 2.50 means able to solve the problem partially, but some gaps remain.
Mean score between 1.00 – 1.50 means unable to solve the problem.

The result from the Expert Evaluation from stated that the experts' opinions regarding the performance of the "AI-powered file organizer application" were overall at the level of able to solve the problem effectively with minor improvements possible, with a mean score of 4.31. The standard deviation was 0.19, indicating a low level of dispersion from the mean. This means that the experts' opinions regarding the application's performance were similar and consistent. And when considering each evaluation item individually, four items were rated at the level of able to solve the problem completely and appropriately. These items were: The system can accurately and appropriately move files to destination folders according to defined templates, The Preview system effectively helps reduce errors before moving or modifying files, The system provides appropriate measures to prevent risks to user data, The system has the potential to support real-world use on Windows. The remaining evaluation items were rated at the level of able to solve the problem effectively with minor improvements possible.

Summary of Expert Opinions on the Performance Evaluation of the "AI-powered file organizer application"

- This application has academic value and can be applied in real-world use. It is recommended that the project be further developed as an Open-Source Project. The developers may also consider publishing a paper on the Hybrid Keyword-AI Approach for Thai-language documents.
- This "AI-powered file organizer application" is highly suitable for presentation at an academic conference. It should also be considered for release as an Open-Source Project so that the community can benefit from it. This would also help build recognition for the institution and the developers.
- The application has a very strong foundation and is ready for personal use or small teams. However, if it is intended for use in large organizations, additional enterprise features should be developed according to the suggestions above. The use of Offline AI is an important strength in the on-premises solution market.

5.3 Project Recommendations

1. Consider adding an adjustable Confidence Score Threshold, such as asking the user for confirmation if the AI confidence score is lower than 70%.
2. Add a Batch Rename feature to rename files according to a predefined format.
3. Consider adding Multi-language Support, starting with Thai and English, which has already been implemented, and possibly adding more languages in the future.
4. Add Export/Import Settings so that users can share templates with their colleagues.
5. Develop a Lite Version for low-specification computers, which may use only keyword matching without AI.
6. Consider Active Directory Integration for organizations using a Windows Domain.
7. Add API Endpoints so that other systems can access and use the application.
8. Develop a Health Check Dashboard for IT administrators to monitor system status.

5.4 Future Work

In addition to the short-term recommendations listed above, the project advisor identified four longer-term directions for extending the application. These are larger in scope than the recommendations above – each one introduces a new capability, a new architectural component, or a new class of file – and are therefore presented separately as future-work items rather than as immediate suggestions.

1. Expanded File-Type Support: Audio and Video

The current version of the application supports text-based documents (PDF, Word, Excel, PowerPoint, TXT) and images. A natural extension is to add support for audio files (such as MP3 and WAV) and video files (such as MP4 and MOV), which are increasingly common in personal and professional file libraries. For audio, a locally runnable speech-to-text model (for example a Whisper-family model) could transcribe the audio content into text, after which the existing content-based categorization logic would apply unchanged. For video, the same approach could be combined with keyframe extraction so that both the visual content (via the image-understanding pipeline) and the spoken content (via transcription) contribute to the categorization decision. This extension would allow the application to organize a much wider range of real-world file libraries without changing its core architecture.

2. Semantic and Contextual Image Understanding

The current image-classification capability operates primarily at the object level, recognizing concepts such as "a cat", "a dog", or "a person." A more powerful future capability is scene-level and event-level understanding – for example, recognizing that a set of photographs depicts "a travel trip with friends in front of historical architecture" rather than merely "people and buildings." This kind of contextual interpretation would allow the application to group personal photo libraries by meaningful events (trips, ceremonies, projects) instead of by isolated objects. The main constraint today is the size and capability of vision-language models that can run fully on-device under the project's offline-first requirement. As smaller, more capable VLMs continue to be released, this capability is expected to become practical to integrate without compromising the local-only architecture.

3. Parallel and Concurrent File Indexing

The current file-indexing pipeline processes files largely in sequence, which is sufficient for the typical workloads tested in this project but does not scale efficiently to libraries containing hundreds of thousands of files. A future iteration could redesign the indexing layer around a parallel, multi-worker model: the file-scanning stage would split the workload across worker threads or processes, each worker would handle a partition of the file tree independently, and a coordinator would aggregate results back into the central database. Operating-system primitives such as thread pools, work-stealing queues, and asynchronous I/O would be used to maximize throughput on multi-core machines while keeping the user interface responsive. This change would substantially reduce the time required to perform a full initial scan of large file libraries.

4. Large-Folder Handling and Automatic Sharding

A known limitation observed in real-world use is that the Windows file system and Windows Explorer become slow or unresponsive when a single folder contains a very large number of files – for example, after a bulk download of 10,000 or more images into the same destination folder. To prevent this from happening as a side effect of automated organization, a future feature could detect when a destination folder is approaching an unsafe file count and automatically shard its contents into sub-folders using a partition key such as the

file's creation date (`2026/05/`), the first letter of the file name (`A/`, `B/`, `C/`, …), or a configurable category-specific scheme. The sharding policy would be exposed in the application's settings so that users can choose the partition key that best matches their workflow, and the existing Preview and Undo system would continue to apply so that any sharding action can be reviewed and reverted.

REFERENCES

1. AWS, 2024, “What is Natural Language Processing (NLP)? - AWS,” Available at https://aws.amazon.com/th/what-is/nlp/, [Online; accessed 4-May-2026].
2. AWS, 2024, “What is LLM? - Large Language Model Explanation - AWS,” Available at https://aws.amazon.com/th/what-is/large-language-model/, [Online; accessed 4-May-2026].
3. NVIDIA, 2024, “What are Vision-Language Models? | NVIDIA Glossary,” Available at https://www.nvidia.com/en-us/glossary/vision-language-models/, [Online; accessed 4-May-2026].
4. AWS, 2024, “What is Retrieval-Augmented Generation (RAG)? - AWS,” Available at https://aws.amazon.com/th/what-is/retrieval-augmented-generation/, [Online; accessed 4-May-2026].
5. Ollama, 2026, “Ollama,” Available at https://ollama.com/, [Online; accessed 4-May-2026].
6. Hugging Face, 2026, “Hugging Face – The AI community building the future,” Available at https://huggingface.co/, [Online; accessed 4-May-2026].
7. Z. Guo, X. Ren, L. Xu, J. Zhang, and C. Huang, 2025, “RAG-Anything: All-in-One RAG Framework,” arXiv:2510.12323 [cs.AI], Oct. 2025. Available at https://arxiv.org/abs/2510.12323, [Online; accessed 4-May-2026].
8. HKUDS, 2026, “GitHub - HKUDS/RAG-Anything: ’RAG-Anything: All-in-One RAG Framework’,” Available at https://github.com/HKUDS/RAG-Anything, [Online; accessed 4-May-2026].
9. HKUDS, 2026, “GitHub - HKUDS/LightRAG: [EMNLP2025] ’LightRAG: Simple and Fast Retrieval-Augmented Generation’,” Available at https://github.com/HKUDS/LightRAG, [Online; accessed 4-May-2026].
10. Python Software Foundation, 2026, “Our Documentation | Python.org,” Available at https://www.python.org/doc/, [Online; accessed 4-May-2026].
11. The Rust Project Developers, 2018, “Documentation - The Rust Programming Language - MIT,” Available at https://web.mit.edu/rust-lang_v1.25/arch/amd64_ubuntu1404/share/doc/rust/html/book/first-edition/documentation.html, [Online; accessed 4-May-2026].
12. Tauri, 2026, “Create a Project - Tauri,” Available at https://v2.tauri.app/start/, [Online; accessed 4-May-2026].
13. D.K. Gifford, P. Jouvelot, M.A. Sheldon, and J.W. O’Toole Jr., 1991, “Semantic File Systems,” in Proc. 13th ACM Symposium on Operating Systems Principles, pp. 16–25, Oct. 1991. Available at https://pages.cs.wisc.edu/~remzi/Classes/838/Fall2001/Papers/sfs-sosp91.pdf, [Online; accessed 4-May-2026].

14. N. Sengar, R.C. Joshi, and M.K. Dutta, 2023, “Auto Organizer: A Machine Learning-Based Tool for Automatic Organization of Files,” Lecture Notes in Electrical Engineering, Jun. 2023. Available at https://www.researchgate.net/publication/371930502_Auto_Organizer_A_Machine_Learning-Based_Tool_for_Automatic_Organization_of_Files, [Online; accessed 4-May-2026].

15. ggml-org, 2026, “ggml-org/llama.cpp: LLM inference in C/C++ - GitHub,” Available at https://github.com/ggml-org/llama.cpp, [Online; accessed 4-May-2026].

APPENDIX A

EXPERTS' FORM

ประวัติผู้เชี่ยวชาญ

ผู้ช่วยศาสตราจารย์ ดร.แสงอุทัย มอโท
อาจารย์ประจำภาควิชาครุศาสตร์อุตสาหกรรม
คณะครุศาสตร์อุตสาหกรรมและเทคโนโลยี
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

แบบประเมินประสิทธิภาพของผู้เชี่ยวชาญต่อ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI”

โครงการ : แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI (AI-powered file organizer application)

สถานศึกษา : มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ชื่อผู้จัดทำโครงการ : นายศรัณย์ คำไทย
นายศุภกร ตั้งปฐมปราโมทย์
นายภูวเดช อินทอง

.....

<u>ส่วนที่ 1</u> คำชี้แจง ใส่เครื่องหมาย (√) ลงในช่องผลการประเมินประสิทธิภาพของ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI” โดยมีความหมายของระดับประสิทธิภาพ มีดังนี้หมายถึง

1 หมายถึง ไม่สามารถแก้ไขปัญหาได้
2 หมายถึง แก้ไขปัญหาได้บางส่วน แต่ยังมีช่องว่างของปัญหาอยู่
3 หมายถึง แก้ไขปัญหาได้อย่างเหมาะสม แต่ยังสามารถปรับปรุงให้ดีขึ้นได้
4 หมายถึง แก้ไขปัญหาได้อย่างมีประสิทธิภาพ หรืออาจปรับปรุงเล็กน้อยได้
5 หมายถึง แก้ปัญหาได้อย่างสมบูรณ์แบบและมีความเหมาะสม รายการประเมิน

ข้อ	รายการประเมิน	ผลการประเมิน				
		1	2	3	4	5
1	ระบบสามารถอ่านและวิเคราะห์ข้อมูลจากไฟล์ PDF, Word, Excel, PowerPoint, TXT และรูปภาพได้อย่างถูกต้อง				√	
2	ระบบสามารถจัดหมวดหมู่ไฟล์ตามเนื้อหาได้อย่างถูกต้องเหมาะสม				√	
3	ระบบสามารถย้ายไฟล์ไปยังโฟลเดอร์ปลายทางตามที่ผู้ใช้กำหนดได้อย่างถูกต้องเหมาะสม					√
4	ระบบ Preview ช่วยลดความผิดพลาดก่อนการย้ายหรือเปลี่ยนแปลงไฟล์ได้จริง					√
5	ระบบบันทึกประวัติและ Undo/Redo มีความเหมาะสมต่อการใช้งานได้จริง				√	
6	การออกแบบ UI มีความง่ายต่อการเรียนรู้และใช้งาน			√		
7	ฟีเจอร์เสริมที่เกี่ยวข้องกับ AI เช่น การสรุปเนื้อหาไฟล์การสร้างโน้ตหรือการค้นหา มีความเหมาะสมต่อการใช้งานจริง				√	
8	ระบบมีแนวทางป้องกันความเสี่ยงต่อข้อมูลผู้ใช้ได้เหมาะสม					√
9	สถาปัตยกรรมระบบโดยรวมมีความเหมาะสมต่อการพัฒนาเป็นเดสก์ท็อปแอปพลิเคชัน				√	
10	ระบบมีศักยภาพในการรองรับการใช้งานจริงบน Windows				√	

<u>ส่วนที่ 2</u> ความคิดเห็น/ข้อเสนอแนะเพิ่มเติมต่อการประเมินประสิทธิภาพของ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI”

แอปพลิเคชันนี้มีคุณภาพอยู่ในระดับดีมาก แสดงให้เห็นถึงความเข้าใจในการประยุกต์ใช้ Local LLM เพื่อแก้ปัญหาจริง มีจุดเด่นหลายประการ ได้แก่:

จุดเด่น:

1. **การใช้ Offline AI** - เป็นจุดเด่นที่สำคัญ เหมาะกับองค์กรที่ต้องการความเป็นส่วนตัว ไม่ต้องพึ่งพา Cloud Service
2. **Preview & Undo System** - ออกแบบมาดีมาก ลดความเสี่ยงจากการจัดไฟล์ผิดพลาด เป็น UX ที่ดี
3. **รองรับภาษาไทยครบถ้วน** - ทั้งการ OCR และ Keyword Matching สำหรับเอกสารภาษาไทย แสดงถึงความใส่ใจในบริบทท้องถิ่น
4. **Hybrid Approach (Keyword + AI)** - การใช้ keyword matching ก่อน แล้วค่อยส่ง AI เป็นแนวคิดที่ดี ช่วยประหยัดเวลาและทรัพยากร

ข้อควรปรับปรุง:

1. **UI/UX** - แม้จะใช้งานได้ แต่ยังดูไม่ทันสมัย แนะนำให้ศึกษา Material Design หรือ Fluent Design เพิ่มเติม อาจพิจารณาเพิ่ม Dark Mode
2. **Error Handling** - ควรมีการจัดการ error ให้ชัดเจนขึ้น เช่น ถ้า LLM ไม่ทำงาน หรือ OCR ล้มเหลว ควรแสดงข้อความที่เข้าใจง่าย
3. **Documentation** - ควรมี User Manual ที่ครบถ้วน รวมถึง Troubleshooting Guide

ข้อเสนอแนะเพิ่มเติม:

- พิจารณาเพิ่ม **Confidence Score Threshold** ให้ผู้ใช้ปรับได้ (เช่น ถ้า AI มั่นใจต่ำกว่า 70% ให้ถามผู้ใช้)
- เพิ่ม **Batch Rename** - จัดชื่อไฟล์ใหม่ตามรูปแบบที่กำหนด

สรุป: แอปพลิเคชันนี้มีคุณค่าทางวิชาการและสามารถนำไปใช้งานจริงได้ แนะนำให้พัฒนาต่อยอดเป็น Open Source Project และอาจพิจารณา Publish Paper เกี่ยวกับ Hybrid Keyword-AI Approach สำหรับเอกสารภาษาไทย

ลงชื่อ ..... ผู้ประเมิน

ผศ.ดร.แสงอุทัย มอโท

ประวัติผู้เชี่ยวชาญ

นายสนธยา เกื้อคีรี

ตำแหน่ง ครู วิทยฐานะ ชำนาญการพิเศษ

แผนกวิชาเทคโนโลยีสารสนเทศ วิทยาลัยเทคนิคตรัง

แบบประเมินประสิทธิภาพของผู้เชี่ยวชาญต่อ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI”

โครงการ : แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI (AI-powered file organizer application)

สถานศึกษา : มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ชื่อผู้จัดทำโครงการ : นายศรัณย์ คำไทย
นายศุภกร ตั้งปฐมปราโมทย์
นายภูวเดช อินทอง

.....

<u>ส่วนที่ 1</u> คำชี้แจง ใส่เครื่องหมาย (√) ลงในช่องผลการประเมินประสิทธิภาพของ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI” โดยมีความหมายของระดับประสิทธิภาพ มีดังนี้หมายถึง

1 หมายถึง ไม่สามารถแก้ไขปัญหาได้
2 หมายถึง แก้ไขปัญหาได้บางส่วน แต่ยังมีช่องว่างของปัญหาอยู่
3 หมายถึง แก้ไขปัญหาได้อย่างเหมาะสม แต่ยังสามารถปรับปรุงให้ดีขึ้นได้
4 หมายถึง แก้ไขปัญหาได้อย่างมีประสิทธิภาพ หรืออาจปรับปรุงเล็กน้อยได้
5 หมายถึง แก้ปัญหาได้อย่างสมบูรณ์แบบและมีความเหมาะสม รายการประเมิน

ข้อ	รายการประเมิน	ผลการประเมิน				
		1	2	3	4	5
1	ระบบสามารถอ่านและวิเคราะห์ข้อมูลจากไฟล์ PDF, Word, Excel, PowerPoint, TXT และรูปภาพได้อย่างถูกต้อง					√
2	ระบบสามารถจัดหมวดหมู่ไฟล์ตามเนื้อหาได้อย่างถูกต้องเหมาะสม				√	
3	ระบบสามารถย้ายไฟล์ไปยังโฟลเดอร์ปลายทางตามที่ผู้ใช้กำหนดได้อย่างถูกต้องเหมาะสม					√
4	ระบบ Preview ช่วยลดความผิดพลาดก่อนการย้ายหรือเปลี่ยนแปลงไฟล์ได้จริง					√
5	ระบบบันทึกประวัติและ Undo/Redo มีความเหมาะสมต่อการใช้งานได้จริง				√	
6	การออกแบบ UI มีความง่ายต่อการเรียนรู้และใช้งาน				√	
7	ฟีเจอร์เสริมที่เกี่ยวข้องกับ AI เช่น การสรุปเนื้อหาไฟล์การสร้างโน้ตหรือการค้นหา มีความเหมาะสมต่อการใช้งานจริง				√	
8	ระบบมีแนวทางป้องกันความเสี่ยงต่อข้อมูลผู้ใช้ได้เหมาะสม					√
9	สถาปัตยกรรมระบบโดยรวมมีความเหมาะสมต่อการพัฒนาเป็นเดสก์ท็อปแอปพลิเคชัน				√	
10	ระบบมีศักยภาพในการรองรับการใช้งานจริงบน Windows					√

<u>ส่วนที่ 2</u> ความคิดเห็น/ข้อเสนอแนะเพิ่มเติมต่อการประเมินประสิทธิภาพของ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI”

แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI นี้มีความโดดเด่นและเป็นประโยชน์อย่างมาก โดยเฉพาะในบริบทการศึกษาและการทำงานจริง

จุดเด่นที่โดดเด่น:

1. **ใช้งานได้จริง** - ไม่ใช่แค่โปรเจกต์ตัวอย่าง แต่สามารถนำไปใช้แก้ปัญหาจริงในสำนักงานหรือหน่วยงานราชการได้ทันที ซึ่งมักมีเอกสารจำนวนมากที่ต้องจัดเก็บ
2. **OCR ภาษาไทย** - นี่คือจุดเด่นมากๆ เพราะเอกสารราชการส่วนใหญ่เป็นภาษาไทย และมักมีเอกสารสแกนที่ต้องทำ OCR การที่รองรับภาษาไทยได้ดีทำให้ใช้งานได้จริง
3. **Offline 100%** - สำคัญมากสำหรับหน่วยงานที่มีข้อมูลสำคัญ ไม่ต้องกังวลเรื่องความปลอดภัย
4. **มี Preview และ Undo** - นักเรียนของผมมักทำโปรเจกต์ที่ไม่คิดถึงจุดนี้ แต่โครงงานนี้คิดถึงผู้ใช้จริงๆ

การประยุกต์ใช้ในการสอน: โครงงานนี้เป็นตัวอย่างที่ดีมากสำหรับนักเรียน นักศึกษา ในการเรียนรู้:

- **Machine learning Integration** – เป็นตัวอย่างการประยุกต์ใช้ Machine learning
- **File Handling** - การจัดการไฟล์หลายฟอร์แมต
- **Real-world Problem Solving** - แก้ปัญหาจริง ไม่ใช่แค่ทำตามตัวอย่าง

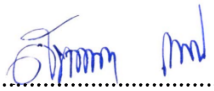
ข้อเสนอแนะในการปรับปรุง:

1. **สร้าง Tutorial Video** - เพื่อให้ผู้ใช้ทั่วไปเข้าใจการใช้งานได้เร็วขึ้น โดยเฉพาะผู้ที่ไม่คุ้นเทคโนโลยี
2. **ปรับ UI ให้ใช้งานง่ายขึ้น** - แม้จะใช้งานได้แล้ว แต่ควรมี Wizard หรือ Guided Setup สำหรับผู้ใช้ครั้งแรก
3. **เพิ่มตัวอย่างการใช้งาน (Use Cases)** - เช่น "จัดเอกสารงานบุคคล", "จัดเอกสารงานการเงิน" เป็น preset template ให้เลือกใช้

ข้อเสนอแนะด้านเทคนิค:

- พิจารณาเพิ่ม **Multi-language Support** - เริ่มจากไทย-อังกฤษก่อน (ทำแล้ว ✓) แต่อาจเพิ่มภาษาอื่นๆ ในอนาคต
- เพิ่ม **Export/Import Settings** - ให้ผู้ใช้สามารถแชร์ template กับเพื่อนร่วมงานได้
- พัฒนา **Lite Version** - สำหรับเครื่องที่มี spec ต่ำ อาจใช้ keyword matching อย่างเดียวโดยไม่ต้องใช้ AI

ความเห็นเพิ่มเติม: แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI นี้เหมาะสมมากสำหรับการนำเสนอในงานประชุมวิชาการ และควรพิจารณาเผยแพร่เป็น Open Source เพื่อให้ชุมชนได้ใช้ประโยชน์ ซึ่งจะเป็นการสร้างชื่อเสียงให้กับสถาบันและผู้พัฒนาด้วย

ลงชื่อ  ผู้ประเมิน

นายสนธยา เกื้อคีรี

ประวัติผู้เชี่ยวชาญ

นายธีรวัฒน์ ณ นคร
หุ้นส่วนผู้จัดการ ห้างหุ้นส่วนจำกัด ทีซีเอ็น เทคโนโลยี
อาจารย์พิเศษ เทคโนโลยีบัณฑิต สาขาเทคโนโลยีสารสนเทศและการสื่อสาร
สถาบันการอาชีวศึกษาภาคใต้ 2 วิทยาลัยเทคนิคตรัง

แบบประเมินประสิทธิภาพของผู้เชี่ยวชาญต่อ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI”

โครงการ : แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI (AI-powered file organizer application)

สถานศึกษา : มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ชื่อผู้จัดทำโครงการ : นายศรัณย์ คำไทย
นายศุภกร ตั้งปฐมปราโมทย์
นายภูวเดช อินทอง

.....

<u>ส่วนที่ 1</u> คำชี้แจง ใส่เครื่องหมาย (√) ลงในช่องผลการประเมินประสิทธิภาพของ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI” โดยมีความหมายของระดับประสิทธิภาพ มีดังนี้หมายถึง

1 หมายถึง ไม่สามารถแก้ไขปัญหาได้
2 หมายถึง แก้ไขปัญหาได้บางส่วน แต่ยังมีช่องว่างของปัญหาอยู่
3 หมายถึง แก้ไขปัญหาได้อย่างเหมาะสม แต่ยังสามารถปรับปรุงให้ดีขึ้นได้
4 หมายถึง แก้ไขปัญหาได้อย่างมีประสิทธิภาพ หรืออาจปรับปรุงเล็กน้อยได้
5 หมายถึง แก้ปัญหาได้อย่างสมบูรณ์แบบและมีความเหมาะสม รายการประเมิน

ข้อ	รายการประเมิน	ผลการประเมิน				
		1	2	3	4	5
1	ระบบสามารถอ่านและวิเคราะห์ข้อมูลจากไฟล์ PDF, Word, Excel, PowerPoint, TXT และรูปภาพได้อย่างถูกต้อง				√	
2	ระบบสามารถจัดหมวดหมู่ไฟล์ตามเนื้อหาได้อย่างถูกต้องเหมาะสม			√		
3	ระบบสามารถย้ายไฟล์ไปยังโฟลเดอร์ปลายทางตามที่ผู้ใช้กำหนดได้อย่างถูกต้องเหมาะสม					√
4	ระบบ Preview ช่วยลดความผิดพลาดก่อนการย้ายหรือเปลี่ยนแปลงไฟล์ได้จริง					√
5	ระบบบันทึกประวัติและ Undo/Redo มีความเหมาะสมต่อการใช้งานได้จริง				√	
6	การออกแบบ UI มีความง่ายต่อการเรียนรู้และใช้งาน			√		
7	ฟีเจอร์เสริมที่เกี่ยวข้องกับ AI เช่น การสรุปเนื้อหาไฟล์การสร้างโน้ตหรือการค้นหา มีความเหมาะสมต่อการใช้งานจริง				√	
8	ระบบมีแนวทางป้องกันความเสี่ยงต่อข้อมูลผู้ใช้ได้เหมาะสม					√
9	สถาปัตยกรรมระบบโดยรวมมีความเหมาะสมต่อการพัฒนาเป็นเดสก์ท็อปแอปพลิเคชัน				√	
10	ระบบมีศักยภาพในการรองรับการใช้งานจริงบน Windows					√

<u>ส่วนที่ 2</u> ความคิดเห็น/ข้อเสนอแนะเพิ่มเติมต่อการประเมินประสิทธิภาพของ “แอปพลิเคชันจัดระเบียบไฟล์อย่างอัตโนมัติด้วย AI”

แอปพลิเคชันนี้จากมุมมองการนำไปใช้งานจริงในองค์กรและความปลอดภัยของข้อมูล

จุดเด่น:

Offline-first Architecture - ข้อมูลไม่ออกนอกเครื่อง เหมาะกับองค์กรที่มีข้อมูลสำคัญ

Local AI Processing - ลดความเสี่ยงด้าน data breach

Password-protected Files Support - คำนึงถึงความปลอดภัยของข้อมูล

System Architecture - Code structure เป็นระเบียบ มี installer ที่พร้อมใช้งาน

ข้อควรปรับปรุงสำหรับ Enterprise Deployment:

Security & Access Control - ควรเพิ่ม user authentication และ role-based access control สำหรับองค์กร

Audit Trail - Log ควรมีรายละเอียดเพิ่มเติม เช่น User ID, IP Address, และ file hash สำหรับ integrity check

Network Deployment - พัฒนา Client-Server Architecture เพื่อให้หลายคนใช้งานร่วมกันได้สะดวก

Backup & Recovery - ควรมีกลไก auto-backup configuration และ export/import settings

ข้อเสนอแนะเพิ่มเติม:

พิจารณา Active Directory Integration สำหรับองค์กรที่ใช้ Windows Domain

เพิ่ม API Endpoint ให้ระบบอื่นๆ สามารถเรียกใช้งานได้

พัฒนา Health Check Dashboard สำหรับ IT Admin ตรวจสอบสถานะระบบ

สรุป: แอปพลิเคชันมีพื้นฐานที่ดีมากและพร้อมใช้งานสำหรับ personal use หรือ small team แต่ถ้าต้องการนำไปใช้ในองค์กรขนาดใหญ่ ควรพัฒนา enterprise features เพิ่มเติมตามข้อเสนอแนะข้างต้น การใช้ Offline AI เป็นจุดเด่นที่สำคัญในตลาด On-premise Solution

ลงชื่อ .....ธีรวัฒน์ ณ นคร..... ผู้ประเมิน

นายธีรวัฒน์ ณ นคร

APPENDIX B

INSTALLATION GUIDE

App Installation Guide

This guide explains how to install KLIN using the Windows setup wizard.

1. Installation Overview

The KLIN setup wizard guides you through five main stages: opening the installer, choosing the destination folder, confirming installation, waiting for the files to install, and finishing the setup. The screenshots in this document match the KLIN Windows installer shown during installation.

2. Requirements

- A Windows computer that can run the KLIN application.
- The KLIN installer file.
- Administrator permission, if Windows requests it during installation.
- Enough disk space in the selected destination folder.

3. Installation Steps

3.1 Open the installer

Run the KLIN setup file. The welcome screen appears. Click **Next** to continue or click **Cancel** if you want to exit the setup wizard.

Figure B.1: Step 1: Welcome to the setup wizard

3.2 Choose the installation folder

The installer shows the default destination folder, usually `C:\Program Files\KLIN\`. To use the default folder, click **Next**. To select a different location, click **Change** and choose another folder.

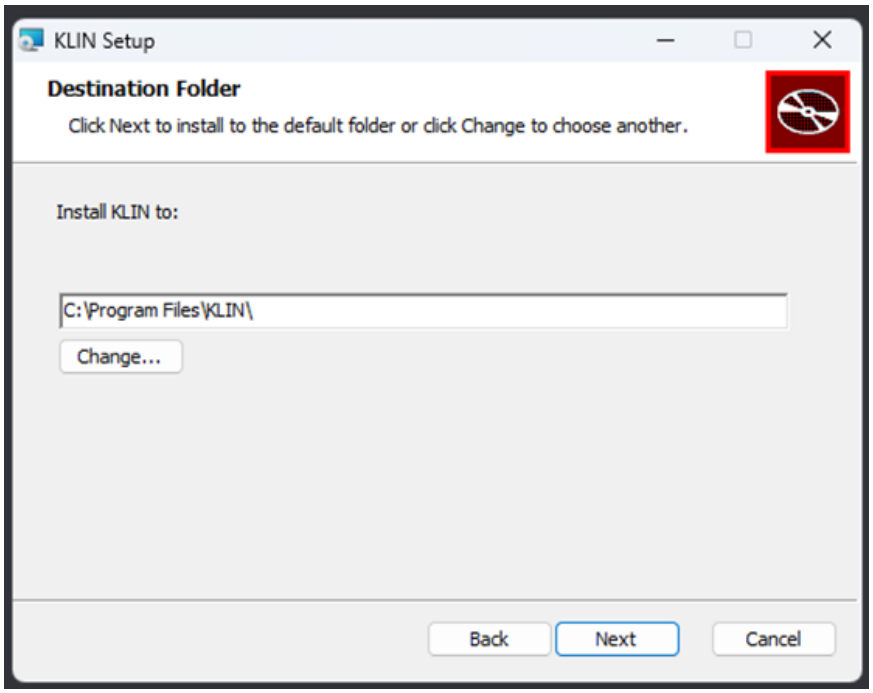


Figure B.2: Step 2: Choose the destination folder

3.3 Confirm installation

When the Ready to install KLIN screen appears, review the installation settings. Click **Install** to begin copying the application files to your computer.

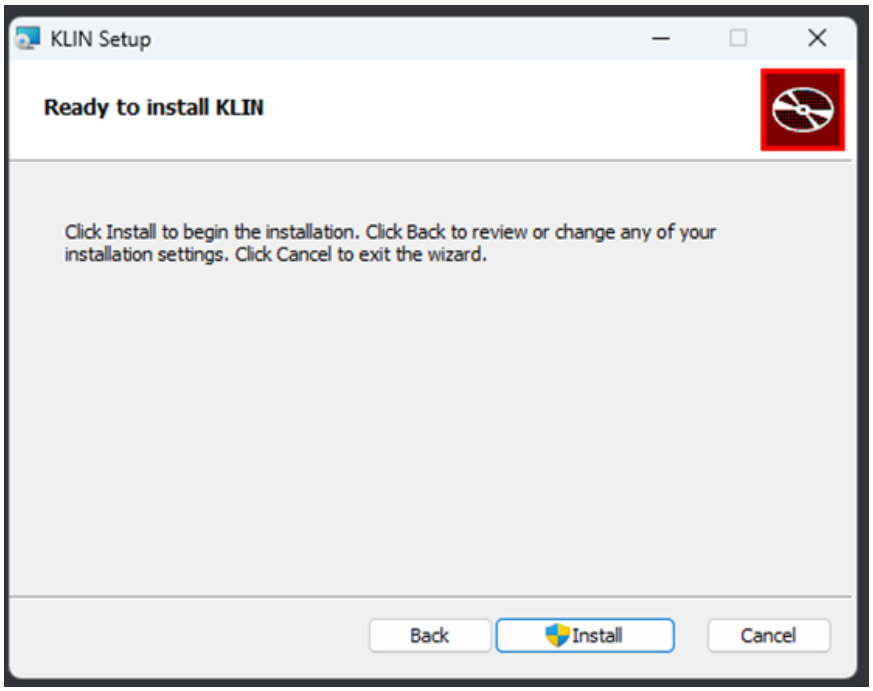


Figure B.3: Step 3: Confirm that KLIN is ready to install

3.4 Wait for the installer

The Installing KLIN screen shows the installation status. Wait until the process finishes. Do not close the setup wizard while installation is in progress.

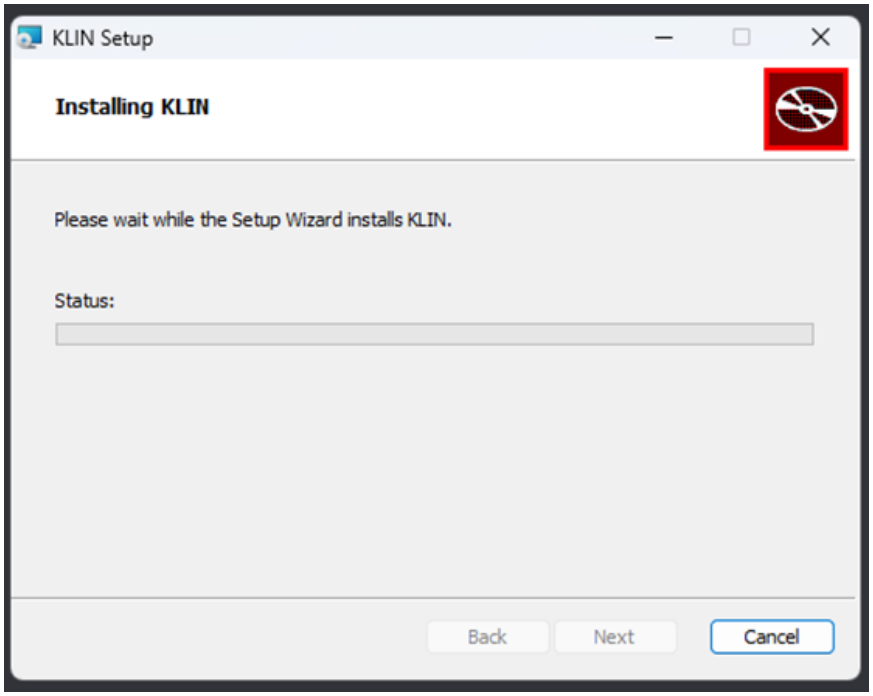


Figure B.4: Step 4: Wait for installation to complete

3.5 Complete the setup

When the Completed the KLIN Setup Wizard screen appears, the installation is finished. Keep **Launch KLIN** checked if you want to open the application immediately, then click **Finish**.

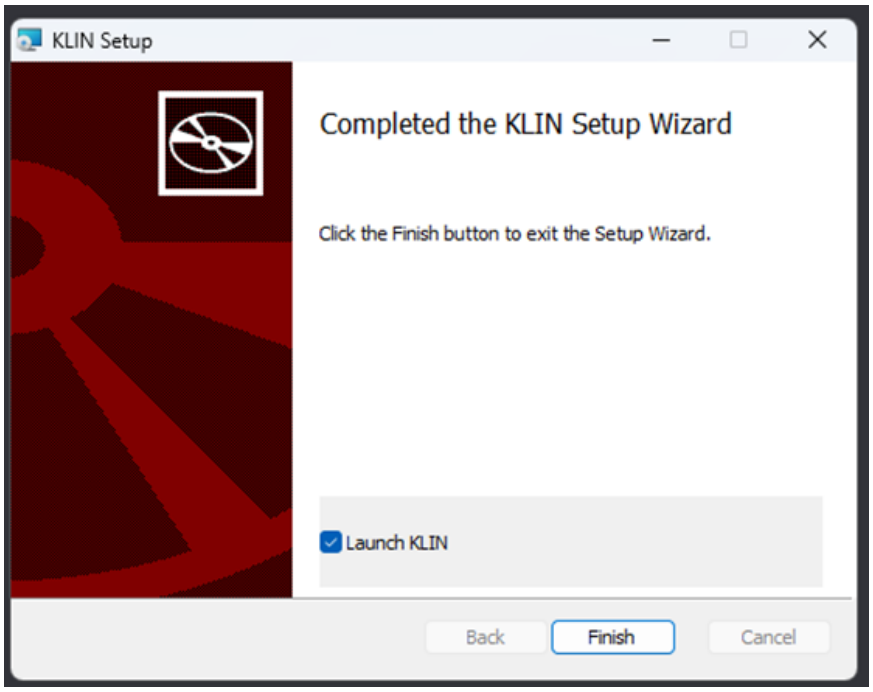


Figure B.5: Step 5: Finish the installation

4. After Installation

If **Launch KLIN** was selected, the application should open automatically after clicking **Finish**. If it does not open automatically, search for KLIN from the Windows Start menu and launch it manually. Check that the application opens correctly before deleting the installer file.

5. Troubleshooting

Issue	Suggested Solution
The installer asks for permission	Click Yes if you trust the KLIN installer and want to continue.
Cannot install to Program Files	Run the installer as administrator or choose another folder using Change.
Installation seems stuck	Wait for a few minutes. If it does not continue, cancel the setup and run the installer again.
KLIN does not launch after Finish	Open KLIN from the Start menu, or restart Windows and try again.